

POLITECNICO DI TORINO

MASTER's Degree in COMPUTER ENGINEERING



MASTER's Degree Thesis

Thesis TITLE

Supervisors

Prof. name SURNAME

Prof. name SURNAME

Prof. name SURNAME

Prof. name SURNAME

Candidate

name SURNAME

DECEMBER 2024

Argumentative Graph-RAG for Participatory Democracy

Michele Pagano Mariano

Abstract

Participatory democracy platforms generate massive volumes of textual data, leading to severe information overload for both citizens and decision-makers. While Natural Language Processing (NLP) and Argument Mining (AM) can extract argumentative structures, navigating this complex data remains a challenge.

This thesis explores the integration of Argument Mining with Retrieval-Augmented Generation (RAG) to create "Graph-RAG." By constructing knowledge graphs that represent the explicit relationships (support, attack, equivalence) between citizen proposals, we aim to enhance the ability of Large Language Models (LLMs) to retrieve context-rich information and generate balanced summaries.

We propose a pipeline that first builds argumentative graphs from civic datasets and then utilizes graph traversal to power a RAG system. This approach addresses the limitations of standard text-based retrieval by exploiting the structural logic of debates, ultimately providing intelligent tools for navigating large-scale deliberative discussions.

ACKNOWLEDGMENTS

Ringrazio la mia famiglia per avermi sempre fatto sentire libero
Ringrazio i miei amici per avermi fatto sentire sempre vivo
Ringrazio me stesso per aver sempre dimostrato che mi sbagliassi

Table of Contents

1	Introduction	1
1.1	Context and Motivation	1
1.2	Problem Statement	2
1.3	Research Questions	2
1.4	Technical Background	3
1.4.1	Natural language processing and argumentative relations. . .	3
1.4.2	Knowledge graphs.	3
1.4.3	Retrieval-augmented generation.	4
1.4.4	Graph-RAG.	4
1.4.5	A Brief History: Why These Techniques Became Prominent .	4
1.4.6	How RAG Works	5
1.4.7	How Graph-RAG Works	6
1.4.8	Advantages and Limitations	6
1.5	Thesis Structure	7
2	Related Work	8
2.1	Argument Mining on Deliberation Platforms	8
2.2	Knowledge Graphs for Argumentative and Deliberative Data	9
2.3	RAG and Graph-RAG in Civic and Governmental Applications . . .	10
3	Corpus and Data	11
3.1	IBM Project Debater Argument Quality Corpus	11
3.2	Descriptive Statistics	12
3.3	Data Cleaning	13
4	Methodology	14
4.1	Graph Construction	14
4.1.1	Policy Grouping: MetaPolicy and Topic	14
4.1.2	Calibrating the CONTRADICTS Threshold	16
4.1.3	Argument-Level Edge Recovery	18
4.1.4	PageRank Centrality	19
4.2	Retrieval Framework	20
4.2.1	Maximal Marginal Relevance	20
4.3	Retrieval Systems Compared	21

4.3.1	StandardRAG: the Embedding-Only Baseline	22
4.3.2	PR-GraphRAG: the Graph-Aware System	22
4.3.3	RAG-no-MMR: a Diversification Ablation	23
4.3.4	Random: a Sanity Floor	24
4.3.5	Summary	24
4.4	Use Cases	25
4.4.1	UC1: Conflict-Map Retrieval	25
4.4.2	UC2: Debate Summarization	25
4.5	Policy Selection	26
4.6	Hardened Pool Construction	27
4.6.1	Why the Native Pool Is Not a Sufficient Test	27
4.6.2	Constructing the Distractor Pool	27
4.6.3	Injection Levels	28
4.6.4	Four Edge-Connectivity Scenarios	29
4.7	Ground Truth Generation	31
4.7.1	Grading Protocol	31
4.7.2	Consensus Resolution	32
4.7.3	Inter-Annotator Agreement	32
4.7.4	Grading the Hardened Pools	33
A	Appendix A	35
B	Appendix B	36
	Bibliography	37
	Dedications	39

List of Figures

List of Tables

1.1	RAG versus Graph-RAG: general properties.	7
3.1	Descriptive statistics of the IBM Project Debater argument quality corpus.	12
4.1	The 10 manually assigned MetaPolicy categories grouping the corpus's 71 policies.	16
4.2	Effect of the bidirectional-minimum criterion on CONTRADICTS edge count, computed over the full 71-policy graph at a blocking floor of 0.70 and threshold of 0.90.	18
4.3	The four retrieval systems compared in this thesis, defined along the two axes MMR leaves open (candidate pool, relevance signal) plus whether diversification is applied.	24
4.4	The four pool-construction scenarios compared in this thesis, by whether injected distractor arguments participate in the CONTRADICTS graph.	30
4.5	Decomposition of the 385,872 CONTRADICTS edges generated over the merged native-plus-distractor argument set at L100, by pair type. . .	30

Acronyms

USB Universal Serial Bus.

Chapter 1

Introduction

1.1 Context and Motivation

Participatory democracy platforms such as Decidim [1], Cap Collectif [2], and Consul [3] enable thousands of citizens to propose and debate public policy. Their adoption has been substantial: Decide Madrid, built on Consul, saw over 430,000 users submit more than 26,000 proposals and 125,000 comments in its citizen-proposals process alone [3]. Similar platforms are now deployed by city and national governments across Europe and Latin America for participatory budgeting, collaborative law-making, and large-scale public consultations such as France’s Grand Débat National.

Their success creates a problem of scale. As the number of contributions grows, no citizen can read the full debate, so they cannot tell which existing proposals resemble or oppose their own, and no organizer can manually review thousands of comments to determine where consensus or disagreement lies. This is precisely the difficulty documented on Consul-based platforms, where both citizens and administrators reported being unable to obtain an overview of the volume of proposals and comments generated [3]. The result is a form of information overload that risks undermining the very inclusiveness these platforms are meant to provide, since a citizen’s contribution is only meaningfully heard if it can be situated within the broader debate, and a debate too large to read is, in practice, a debate too large to participate in.

Prior work has shown that some of this structure can be recovered automatically. Natural language processing models can detect argumentative relations between citizen proposals, determining whether two proposals make the same point or whether they are in direct conflict, and these relations can be organized into an argumentative graph that makes the shape of the debate explicit rather than leaving it buried in unstructured text. Helping a user see this shape, meaning the relevant arguments on both sides of an issue without redundancy, is fundamentally a retrieval problem: given a policy under debate, which arguments should be surfaced, and how should they be organized so that the retrieval itself does not reintroduce the very redundancy and imbalance it is meant to resolve?

1.2 Problem Statement

The preceding section motivates the need to make the structure of a large-scale civic debate explicit rather than leaving it as an unstructured collection of proposals. The remaining question is how a retrieval system should exploit that structure once it exists.

The default approach, dense retrieval over text embeddings, treats each argument as an independent point in a vector space and retrieves the arguments closest to a query. This is effective at finding topical relevance, but it has no notion of two arguments being redundant restatements of the same point, nor of two arguments being in direct tension. A user asking to see the debate on a policy is not well served by five arguments that all reflect the same view in different words, nor by a set of pro and con arguments that never actually engage the same underlying claim. Recovering this structure, which arguments repeat each other and which arguments conflict, requires relations between arguments, not just positions in an embedding space.

Graph-based retrieval methods (Graph-RAG) address this by building an explicit graph over the corpus and using its structure, rather than embedding distance alone, to guide retrieval. Whether this actually improves retrieval for argumentative content is, however, an open empirical question rather than an established result: the graph adds a strong inductive bias (that structurally connected arguments are the ones worth retrieving together), and inductive biases help only when they match the true structure of the task. This thesis investigates that question directly. Concretely, it builds an argument graph over a large debate corpus, with edges representing contradiction and equivalence between arguments, and compares retrieval that exploits this graph structure against retrieval that relies on embedding similarity alone, across two tasks: retrieving a balanced, non-redundant, and genuinely conflicting set of arguments for a policy (conflict-map retrieval), and producing a debate summary from the retrieved arguments (debate summarization).

1.3 Research Questions

The problem statement above is investigated through three research questions, each targeting a distinct threat to the validity of a simple positive answer.

RQ1 (Does structure help). For conflict-map retrieval (UC1), does a graph-structure-aware retrieval system, one that expands its candidate pool through explicit CONTRADICTS relations and ranks by graph centrality, achieve higher relevance and diversity than a system that relies on embedding similarity alone, and does any such advantage survive into debate summarization (UC2)?

RQ2 (Is the advantage genuine). A relevance or diversity gap measured on a corpus’s native pool can reflect genuine retrieval quality, or it can reflect properties of the evaluation setup itself, such as which candidates happen to populate the pool and how they were selected. Under an evaluation protocol

deliberately hardened with adversarial distractors, does any advantage observed under RQ1 persist, and is it attributable to the retrieval method rather than to how the pool was constructed?

RQ3 (Does the finding generalize). The corpus used to answer RQ1 and RQ2 is a single English-language, crowd-authored dataset. Does the same pattern of results hold on an independently constructed corpus of real citizen contributions in a different language, namely the French Grand Débat National, or is it specific to the properties of the original corpus?

RQ1 is answered in Chapter ?? through the UC1 and UC2 evaluations. RQ2 is answered by two robustness checks built into the same evaluation protocol: injecting distractor arguments with their own graph edges, and reconstructing the candidate pool under random rather than similarity-based selection. RQ3 is answered by repeating the UC1 pipeline on the Grand Débat National corpus described in Chapter 3. Taken together, these three questions are designed so that a positive answer to RQ1 alone would not be sufficient to claim that graph structure improves retrieval: the claim is only supported if it also survives RQ2 and, ideally, RQ3.

1.4 Technical Background

This thesis draws on four strands of prior work, each contributing one layer of the system built here.

1.4.1 Natural language processing and argumentative relations.

Determining whether one piece of text supports, restates, or contradicts another is a core NLP problem, most commonly framed as natural language inference (NLI): given a premise and a hypothesis, predict whether the hypothesis is entailed, contradicted, or neutral with respect to the premise. Argument mining specializes this general machinery to argumentative text specifically, extracting claims, premises, and the relations between them from unstructured documents [4]. Prior work has shown that such relations, including direct attack and support between arguments, can be identified automatically using supervised learning over textual features [cocarascu2017], and that entailment-style reasoning transfers naturally to the argumentative setting, where two arguments can be treated as equivalent, contradictory, or unrelated depending on their entailment relationship in each direction [cabrio2012]. This thesis relies on the same underlying idea, using NLI as the scoring mechanism that produces the CONTRADICTS and EQUIVALENT relations described in Chapter 4.

1.4.2 Knowledge graphs.

A knowledge graph represents entities as nodes and their relations as typed edges, making relational structure explicit and queryable rather than leaving it implicit in text or buried in independent embeddings [5]. The corpus used in this thesis is

organized exactly this way: arguments are nodes, and CONTRADICTS/EQUIVALENT relations are typed edges between them, forming an argumentative knowledge graph. Graph structure of this kind supports operations that are awkward or impossible with a flat set of embeddings alone, such as measuring how central an argument is within the debate through graph centrality, or finding the community of arguments that all restate the same underlying point through community detection.

1.4.3 Retrieval-augmented generation.

Retrieval-augmented generation (RAG) combines a retriever, which selects relevant items from a corpus given a query, with a generator, typically a large language model, that conditions its output on the retrieved items [6]. RAG was originally motivated by knowledge grounding, letting a generator produce answers backed by retrieved evidence rather than relying solely on parametric memory, and standard RAG retrieves items independently by embedding similarity: each candidate is scored against the query in isolation, with no notion of how candidates relate to one another.

1.4.4 Graph-RAG.

Graph-RAG extends this paradigm by making the retriever graph-aware: rather than treating candidates as independent points in an embedding space, it exploits explicit relations between them, using graph traversal, centrality, or community structure to decide what to retrieve and how to diversify it [7]. This matters directly for the retrieval problem posed in Section 1.2: standard RAG can retrieve five arguments that are all close to a query in embedding space without ever detecting that they restate the same point, since embedding similarity alone does not distinguish topical closeness from redundancy or conflict. A graph-aware retriever can use EQUIVALENT edges to avoid retrieving restatements of the same argument, and CONTRADICTS edges to actively seek out arguments in genuine tension, which is precisely the conflict-map retrieval task (UC1) this thesis evaluates. Whether this theoretical advantage translates into a measurable empirical improvement over standard RAG, on a real argumentative corpus rather than the general-domain Wikipedia-style corpora typically used to evaluate Graph-RAG, is the question this thesis investigates.

1.4.5 A Brief History: Why These Techniques Became Prominent

RAG and Graph-RAG are recent enough that understanding why they became prominent, rather than only what they do, is useful context for the design choices made later in this thesis.

The RAG architecture itself was proposed in 2020 as an academic technique for knowledge-intensive question answering [6], but it remained a comparatively specialized topic for roughly three years. Its move from a research technique to a widely adopted engineering practice followed the public release of general-purpose conversational LLMs in late 2022, whose fluency made a specific weakness visible at scale: these models generate confident, well-formed answers that can nonetheless be

factually wrong, a failure mode generally termed hallucination. A model’s parametric knowledge is also frozen at training time and cannot easily incorporate new or private information without retraining. RAG addresses both problems at once: conditioning generation on retrieved passages gives the model something to ground its answer in, and updating the corpus updates the model’s effective knowledge without touching its parameters at all. By 2023, retrieval had become a standard component of production LLM systems, and RAG in particular had gone from a specialized technique to one of the most widely deployed patterns for building LLM applications [8].

Graph-RAG emerged as a response to a specific limitation of this now-standard RAG pattern: flat vector retrieval struggles with questions that require synthesizing information spread across many documents, rather than answerable from any single retrieved passage, since similarity search surfaces individually relevant items without any notion of how those items relate to each other collectively. Microsoft Research’s Graph-RAG system, which builds a knowledge graph over a corpus and summarizes at the level of detected communities to answer such global, corpus-spanning questions, was an influential proposal in 2024 for addressing this gap [7] and popularized the term “Graph-RAG” for the broader family of graph-structured retrieval approaches that has since grown around it.

For this thesis, the significant part of this history is what it implies about maturity. RAG is well-established, with a large literature on retriever design, chunking, and evaluation, so System A in this thesis builds on settled practice. Graph-RAG is comparatively young, its empirical evaluations concentrate on general-domain, entity-centric corpora such as Wikipedia-derived text, and rather less is established about how it performs on argumentative text, where the relations that matter (contradiction and equivalence between claims) are not the entity co-occurrence relations that most existing Graph-RAG systems are built around. This is the specific gap this thesis addresses.

1.4.6 How RAG Works

A RAG system has two components. The *retriever* takes a query and returns the k items from a corpus most relevant to it, typically by embedding the query and every corpus item into a shared vector space and ranking candidates by cosine similarity [6]. The *generator*, usually a large language model, then conditions its output on both the original query and the retrieved items, rather than relying only on knowledge stored in its parameters. This separation is the central idea: retrieval supplies up-to-date, specific, and verifiable content, while generation supplies fluent, task-adapted language production. Because retrieval and generation are decoupled, the corpus can be updated without retraining the generator, and the system’s outputs can, in principle, be traced back to the specific items retrieved.

A well-known limitation of naive top- k retrieval is redundancy: the k most similar items to a query are often near-duplicates of each other, since text that is highly similar to the query tends also to be highly similar to other similarly-relevant text.

Maximal marginal relevance (MMR) addresses this by re-ranking candidates to jointly maximize relevance to the query and dissimilarity to items already selected [9], trading off some relevance for broader coverage of the retrieved set. This diversity/relevance tradeoff, and the metrics used to quantify it, recur throughout this thesis’s evaluation methodology (Chapter ??).

1.4.7 How Graph-RAG Works

Graph-RAG replaces or augments the flat vector index with an explicit graph over the corpus, where nodes are corpus items and edges encode a relation between them, such as citation, co-reference, shared entities, or, in this thesis, argumentative agreement and disagreement. Retrieval then exploits this structure directly. Two broad strategies are common in the literature. The first constructs communities of densely connected nodes and, at query time, retrieves or summarizes at the level of communities rather than individual items, letting the system answer questions that require synthesizing information spread across many related items rather than any single one [7]. The second, closer to the approach used in this thesis, expands a candidate set by traversing edges from an initial seed, so that items connected to a highly relevant node are considered even if they are not themselves close to the query in embedding space, and ranks the resulting candidates using graph-centrality measures such as PageRank in addition to, or instead of, embedding similarity.

The practical payoff is that Graph-RAG can retrieve based on *relationships* that embedding similarity does not capture. Two arguments that directly contradict each other are not necessarily close in embedding space, since surface wording can differ substantially even when the underlying claims are in direct tension; conversely, two arguments can be embedding-close while being logically unrelated, simply because they discuss the same topic. A graph edge encodes the relationship explicitly, rather than leaving the retriever to infer it indirectly from distance in a vector space.

1.4.8 Advantages and Limitations

Both paradigms have genuine strengths and corresponding weaknesses, summarized in Table 1.1.

Standard RAG is simple to build and maintain: it requires only an embedding model and a vector index, degrades gracefully as the corpus grows, and needs no relation-extraction step. Its weakness is exactly what motivates Graph-RAG: it has no mechanism for reasoning about how candidates relate to one another, so redundancy and conflict must be inferred indirectly, if at all, from embedding geometry.

Graph-RAG’s strength mirrors this weakness: with a well-constructed graph, it can retrieve based on explicit relational signal that embedding similarity structurally cannot represent. Its cost is that this strength is entirely conditional on graph quality. Building the graph requires a relation-extraction step (Section 1.4.1) that can introduce its own errors, an under-connected or noisy graph provides little advantage over standard RAG, and the additional machinery, edge construction,

graph traversal, centrality computation, adds engineering and computational cost that standard RAG does not incur. Whether this cost is worth paying is an empirical question that depends on the corpus and the task, not a foregone conclusion in Graph-RAG’s favor, which is precisely the question this thesis investigates for argumentative retrieval.

Table 1.1: RAG versus Graph-RAG: general properties.

Property	Standard RAG	Graph-RAG
Candidate comparison	Each candidate scored against the query independently	Candidates can be scored relative to one another via graph structure
Redundancy handling	Indirect, via embedding-distance heuristics (e.g. MMR)	Direct, via explicit relation edges (e.g. equivalence)
Relational reasoning	None; relationships must be inferred from geometry	Explicit; relationships are given as typed edges
Setup cost	Low: embedding model and vector index	Higher: requires a relation-extraction pipeline
Failure mode	Redundant or topically narrow results	Degrades to no better than standard RAG if the graph is sparse or noisy

1.5 Thesis Structure

Chapter 2

Related Work

2.1 Argument Mining on Deliberation Platforms

Elguendouze et al. [10] present AM4DSP, a system that performs argumentation mining directly on structured decentralized discussion platforms built for deliberative democracy, classifying statements into position and evidence roles and identifying relations between them, evaluated on the BCause platform. This is close precedent for the relation-extraction stage of this thesis: both projects mine argumentative structure from real or realistic civic discussion data rather than curated debate corpora. The key divergence is scope. AM4DSP targets *discussion-level* analysis within a single deliberation thread, structuring the argumentation inside one conversation; this thesis instead asks a *retrieval* question across an entire corpus of thousands of independent arguments spanning many policies, where the challenge is selecting and organizing a relevant, non-redundant, conflicting subset rather than parsing the structure of one discussion. AM4DSP’s mined relations are consumed by human moderators for sense-making; the relations extracted in this thesis are consumed by an automated retrieval and summarization pipeline evaluated against graded relevance judgments, a different downstream use that motivates a different evaluation protocol entirely (Chapter ??).

Anastasiou and De Liddo [11] describe BCause, a human-AI collaboration tool that supports hybrid mapping and ideation in argumentation-grounded deliberation, funded under the EU ORBIS programme on augmenting trust and transparency in deliberative democracy. Like this thesis, BCause is built on the premise that making argumentative structure explicit improves a deliberation process; unlike this thesis, BCause is fundamentally a human-in-the-loop authoring and mapping tool, where a facilitator or participant actively builds and edits the argument map. This thesis instead evaluates a fully automated retrieval pipeline with no human curation step at query time, trading the higher precision a human curator could provide for the ability to operate at the scale of a 30,500-argument corpus, where manual mapping is not feasible.

Behrendt et al. [12] survey the broader landscape of machine-learning methods for enhancing deliberation in online political discussions, cataloguing which tasks

current AI methods address and identifying open challenges. Their survey confirms that argument-relation detection and redundancy/conflict identification, the two relation types this thesis constructs, are recognized open problems in the deliberation-technology literature rather than a solved prerequisite; this motivates treating the calibration of these relations (Chapter 4) as a contribution in its own right, not a routine preprocessing step.

2.2 Knowledge Graphs for Argumentative and Deliberative Data

Plenz et al. [13] construct PAKT, a perspectivized argumentation knowledge graph built directly for deliberation analysis, encoding not only argumentative relations but the perspective, or stance, from which each argument is made. PAKT’s graph construction is close in spirit to this thesis’s own CONTRADICTS/EQUIVALENT edges, and its explicit handling of perspective anticipates a challenge this thesis meets independently on the Grand Débat National corpus, where stance and topic are shown to be entangled in embedding space (Section ??). The divergence is again one of downstream purpose: PAKT is positioned as an analysis and visualization tool for researchers studying deliberation, not as the retrieval backend for a conflict-mapping or summarization system, and its evaluation is correspondingly framed around graph quality and analytical utility rather than retrieval metrics such as nDCG or diversity.

Vagnoni and Rodríguez-Doncel [14] introduce the Deliberation Knowledge Graph, integrating deliberation data from European Parliament proceedings, civic participation platforms, and public forums under a shared ontology, and demonstrate its use for examining argument quality and tracing the evolution of policy positions across institutional and civic spheres. This work shares this thesis’s premise that an explicit graph over argumentative data supports analysis that flat text cannot, but it is oriented toward *integration* across heterogeneous, multi-source institutional data under a common ontology, whereas this thesis constructs a single-source, within-corpus graph and asks a comparative retrieval question (graph-aware versus embedding-only retrieval) that the Deliberation Knowledge Graph’s descriptive, ontology-integration framing does not address.

“DebateKG: Automatic Policy Debate Case Creation with Semantic Knowledge Graphs” [15] build argumentative semantic knowledge graphs over a large policy-debate corpus and construct debate cases via constrained shortest-path traversal over the resulting graph, evaluated on an extension of the DebateSum dataset, a close relative of the IBM Project Debater corpus used in this thesis. DebateKG is the closest existing work to this thesis in terms of *data domain*: both operate on competitive or crowd-authored policy-debate text at comparable scale. The divergence is in task and evaluation. DebateKG’s shortest-path traversal is optimized to construct a single coherent debate case (a chain of connected arguments serving one side), not to retrieve a balanced, graded-relevant, non-redundant set spanning

both PRO and CON stances for a conflict map, and DebateKG reports no comparison against an embedding-only baseline, leaving open exactly the empirical question, whether graph traversal outperforms dense retrieval on this class of corpus, that this thesis investigates directly (RQ1, Section 1.3).

2.3 RAG and Graph-RAG in Civic and Governmental Applications

Papageorgiou et al. [16] propose a multi-agent GraphRAG framework for e-government, motivated explicitly by the observation that flat, unstructured RAG retrieval limits multi-hop reasoning and interpretability on structured government data, and demonstrate the framework on a policy-focused question-answering case study. This is the closest existing work to this thesis’s core empirical question, a direct application of the RAG-versus-Graph-RAG comparison to policy text. It differs from this thesis in three respects that matter for what can be concluded from it. First, its graph is built over policy *documents* (structured e-government datasets, press releases), not over individual *citizen arguments*, so its edges represent document or entity relations rather than the contradiction and equivalence relations between argumentative claims that this thesis’s retrieval systems exploit. Second, its evaluation is a single case study demonstrating the framework’s operation, not a controlled comparison against a matched embedding-only baseline under a shared evaluation protocol, so no quantitative claim is made about whether the graph-aware component actually outperforms flat retrieval on this data, the precise gap this thesis’s UC1 evaluation is designed to close. Third, it does not evaluate a diversity or redundancy criterion, whereas avoiding redundant retrieval is central to this thesis’s conflict-map task (UC1). Taken together, this thesis can be read as supplying the controlled, quantitative retrieval evaluation that this line of applied GraphRAG work motivates but does not itself provide.

Tessler et al. [17] present the Habermas Machine, an LLM-based system that generates group statements intended to represent common ground across a set of citizen opinions, evaluated in large-scale deliberation experiments. While not a retrieval system and not graph-based, this work establishes that LLM-mediated systems can measurably improve deliberative outcomes at scale, which motivates this thesis’s underlying premise that the retrieval stage feeding such a generation system is worth optimizing carefully; a summarization or common-ground system is only as good as the arguments it is given to work with, a claim this thesis tests directly in UC2, where retrieval-quality differences are evaluated for whether they survive the summarization bottleneck (Section ??).

Chapter 3

Corpus and Data

3.1 IBM Project Debater Argument Quality Corpus

The primary corpus used in this thesis is the argument quality ranking dataset released as part of the IBM Project Debater initiative **gretz2020argq**. The dataset consists of approximately 30,500 short arguments distributed across 71 controversial policy topics, each phrased as a debate motion such as “We should ban homeschooling” or “We should subsidize Wikipedia.” Arguments were collected through crowdsourcing: annotators were shown a topic and asked to contribute a short argument either supporting or opposing it, so every argument in the corpus is authored text rather than an excerpt extracted from a longer document. This authorship model gives the corpus a property that matters for the retrieval task studied later in this thesis: each argument is already a self-contained, single-claim unit of roughly sentence-to-short-paragraph length, which removes the need for a separate argument-segmentation step and lets the retrieval and graph-construction pipeline (Chapter 4) operate directly on the corpus’s native units.

Every argument carries two independent crowd-annotated labels, both of which this thesis relies on directly. The first is a *stance* label, recording whether the argument supports or opposes its associated topic. Stance was collected from multiple annotators per argument and aggregated into a weighted-average score together with a confidence value, rather than taken as a single annotator’s judgment; arguments where annotators disagreed enough to leave the aggregated stance close to the decision boundary are distinguishable from unambiguous cases through this confidence value. The second is a *quality* label, produced through a large-scale pairwise and rating-based crowdsourcing protocol in which annotator reliability was estimated and low-reliability judgments down-weighted using MACE [hovy2013mace], yielding a per-argument quality score together with an associated confidence estimate. Quality, in this annotation scheme, reflects how convincing or well-formed an argument is judged to be independent of which side it takes, and is separate from the stance label. This thesis makes direct use of the stance label to construct the graph’s argument-to-policy edges: a PRO argument is linked to its policy through a SUPPORTS edge and a CON argument through an ATTACKS edge (Section 4.1). The quality score is not

used as a graph feature but is retained in the corpus and available as a covariate for later analysis.

Topic coverage in the corpus is narrow rather than skewed in the way one might expect. The number of arguments per policy ranges from 331 to 554 (Table 3.1), so no policy is underrepresented relative to the others by an order of magnitude; this still leaves a roughly 1.7-fold difference between the sparsest and densest policy, which is the property that motivates the stratified policy-selection step described in Chapter 4. Stance balance, by contrast, is remarkably consistent across policies: as Section 3.2 shows, every one of the 71 policies has a PRO ratio between 47% and 55%, so unlike the argument-count range, stance imbalance is not a property this corpus exhibits and is not a consideration behind the policy-selection step.

One property of the corpus places a boundary around what this thesis can claim. Arguments were elicited directly for the purpose of building this dataset rather than drawn from an existing deliberation platform, so their length, register, and single-claim structure are more uniform than what a real citizen-participation corpus produces, where contributions vary widely in length, coherence, and how directly they engage with the policy under discussion.

3.2 Descriptive Statistics

Table 3.1 summarizes the corpus after the cleaning step described in Section 3.3; in this case cleaning removed no rows, since no argument fell below the dataset’s minimum length threshold and every argument received a non-neutral stance label.

Table 3.1: Descriptive statistics of the IBM Project Debater argument quality corpus.

Statistic	Value
Total arguments	30,497
Policies (topics)	71
PRO arguments	15,448
CON arguments	15,049
Neutral-stance arguments	0
Arguments per policy (min / median / max)	331 / 421 / 554
Arguments per policy (mean, std)	429.5, 45.9
Per-policy PRO ratio (min / median / max)	0.475 / 0.505 / 0.542
Policies with PRO ratio outside [40%, 60%]	0 / 71
Argument length in tokens (mean, std)	18.2, 7.5
Argument length in tokens (median)	17
Arguments with stance confidence below 0.7	1,135

Overall the corpus is close to stance-balanced, with PRO arguments outnumbering CON arguments by only 399 out of 30,497, and this balance holds at the policy level as well: the per-policy PRO ratio ranges only from 0.475 to 0.542, and all 71 policies fall within a 40%–60% band. The policy with the strongest PRO skew, on the abolition of nuclear weapons, still has 254 CON arguments against 300 PRO; the

policy with the strongest CON skew, on adopting atheism, has 199 CON arguments against 180 PRO. Stance skew is therefore not a property this corpus exhibits at either the corpus-wide or the policy level, and is not a factor behind the policy-selection step in Chapter 4, which instead responds to the more modest variation in argument count per policy. Stance-confidence is high throughout, with only 1,135 arguments (3.7% of the corpus) falling below a 0.7 confidence threshold, indicating that crowd annotators largely agreed on the stance of each argument. Argument length is also fairly uniform, averaging 18.2 tokens (whitespace-delimited) with a standard deviation of 7.5, consistent with the corpus’s single-sentence, single-claim authorship model described in Section 3.1.

3.3 Data Cleaning

Before use, the raw corpus underwent a light cleaning pass: fields were renamed for consistency with the rest of the pipeline (the topic field, in particular, is referred to as *policy* throughout this thesis), and arguments were checked against the dataset’s own minimum length threshold and for malformed or empty strings. On this corpus these checks removed nothing: as Table 3.1 shows, all 30,497 arguments passed cleaning intact, so the checks function as a safeguard against malformed input rather than as a meaningful filtering step. This step is deliberately minor and does not alter the crowd-assigned stance or quality labels; the substantive preprocessing this corpus undergoes, recovering the CONTRADICTS and EQUIVALENT relations between arguments through an NLI cross-encoder, is treated as a methodological contribution in its own right and is described in Chapter 4 rather than here.

Chapter 4

Methodology

4.1 Graph Construction

The retrieval systems compared in this thesis operate over an explicit graph built on top of the IBM Project Debater corpus described in Chapter 3, rather than over a flat collection of embedded arguments. At its core, the graph has two node types, policies and arguments, connected by edges of four types. SUPPORTS and ATTACKS edges connect an argument to the policy it addresses, following directly from the corpus’s crowd-annotated stance label: a PRO argument receives a SUPPORTS edge to its policy and a CON argument an ATTACKS edge, each carrying the annotation’s stance confidence as an edge weight. CONTRADICTS and EQUIVALENT edges connect pairs of arguments to each other, and, unlike the stance edges, are not present in the corpus and have to be recovered. CONTRADICTS edges are the backbone of the graph-aware retrieval system’s multi-hop expansion (Section 4.3); EQUIVALENT edges are constructed for the same argument pairs but play a smaller role in this thesis’s retrieval systems, and are retained chiefly as a resource for identifying near-duplicate arguments in the corpus. Above the policy level, two further node types, MetaPolicy and Topic, group policies hierarchically rather than argument by argument; this grouping is described next and is used later in the hardened pool construction (Section 4.6) to identify topically-adjacent sibling policies for a target policy.

4.1.1 Policy Grouping: MetaPolicy and Topic

The 71 policies in the corpus are not treated as a flat, unordered list. Two coarser grouping levels sit above them: a BELONGS_TO edge connects each Policy node to one of 10 hand-labeled MetaPolicy nodes, and each MetaPolicy node in turn connects via its own BELONGS_TO edge to one of 5 Topic nodes. This hierarchy was introduced because several downstream steps need a principled notion of which policies are topically close to a given target policy without being the same policy, most directly the sibling-policy selection used to construct hardened, distractor-injected evaluation pools (Section 4.6): a distractor argument drawn from a policy in the same MetaPolicy as the target (for example, drawing a hard negative for “We should ban cosmetic surgery” from the sibling policy “We should legalize organ trade”,

both under Bioethics & Personal Autonomy) shares vocabulary and framing with the target policy without answering it, which is precisely what makes it a useful adversarial test rather than a trivially off-topic one.

Grouping the 71 policies was attempted first as an unsupervised clustering problem, before falling back to manual assignment, following four steps:

1. **Centroid computation.** Each policy was represented by a single centroid vector, the mean of the SENTENCE-TRANSFORMERS/ALL-MPNET-BASE-V2 embeddings [reimers2019sbert, song2020mpnet] of that policy’s own PRO and CON arguments, re-normalized to unit length after averaging since the mean of several unit vectors does not itself have unit norm. This gives a 71×768 centroid matrix, one row per policy.
2. **Dimensionality reduction.** The centroid matrix was reduced to 20 dimensions with PCA to denoise it before clustering, rather than clustering directly in the full 768-dimensional space.
3. **k -means sweep.** k -means was run on the reduced space for every k from 5 to 15, each run scored by silhouette coefficient.
4. **HDBSCAN sweep.** HDBSCAN [campello2013hdbscan] was run on the same reduced space with its minimum cluster size ranging from 3 to 6, each run scored by silhouette coefficient over the non-noise points.

Neither method produced a clean, well-separated partition:

- **k -means** silhouette scores stayed in a narrow and modest band across the entire swept range, peaking around $k = 9$ – 10 at a score of roughly 0.14, well below the 0.5-plus range conventionally read as indicating real cluster structure.
- **HDBSCAN** was, if anything, less decisive: at a minimum cluster size of 6 it labeled every one of the 71 policies as noise and returned no clusters at all, and even at the smaller minimum cluster sizes that did return clusters it found only 2–4 of them, with 55–79% of policies left unclustered as noise.
- **Agreement between the two.** The two methods broadly agreed on the order of magnitude of a defensible cluster count, roughly single digits to the low teens, but neither converged on a specific partition. The weak silhouette scores throughout indicate that the 71 policies do not separate into discrete regions of embedding space so much as blend into one another, which is unsurprising given that many crowd-authored debate topics share vocabulary or framing across what are, substantively, different policy questions.

Given the absence of a numerically decisive clustering solution, the final MetaPolicy grouping was assigned manually: the candidate k -means and HDBSCAN partitions were used as a reference point, but each of the 71 policies was ultimately placed into one of 10 hand-labeled MetaPolicy categories (Table 4.1) by inspecting the

policy texts directly for substantive thematic overlap, rather than by taking either algorithm’s output as final. Scored on the same reduced embedding space, this manual assignment obtains a silhouette score of 0.09, lower than k -means’ best automatic partition; this is expected rather than a failure of the manual step, since the manual assignment optimizes for a human-legible, substantively coherent category scheme rather than for geometric separation in embedding space, and the two criteria need not agree given how weakly separated the policies are to begin with. The 10 MetaPolicies were then grouped, again manually, under 5 broader Topic categories (Politics & Governance, Law & Justice, Health & Ethics, Economy & Society, and Social & Cultural Issues), giving the three-level Topic / MetaPolicy / Policy hierarchy used throughout the rest of this thesis.

Table 4.1: The 10 manually assigned MetaPolicy categories grouping the corpus’s 71 policies.

ID	MetaPolicy
1	Political Governance
2	Criminal Justice & Law
3	Civil Liberties & Rights
4	Bioethics & Personal Autonomy
5	Economic & Labor Policy
6	Religion & Secularism
7	Gender, Family & Social Norms
8	Education, Media & Technology
9	Animal Welfare & Environment
10	Child & Family Welfare

4.1.2 Calibrating the Contradicts Threshold

Recovering CONTRADICTS and EQUIVALENT edges is framed as a natural language inference (NLI) problem, following the argumentation literature reviewed in Section 1.4.1: two arguments contradict each other if each entails the other’s negation, and are equivalent if each entails the other directly. An initial attempt at this idea accepted any candidate pair whose forward contradiction probability alone exceeded a fixed threshold, and produced a graph in which roughly a third of all possible cross-stance pairs within a policy were connected by a CONTRADICTS edge. Inspecting this result showed the underlying problem: single-direction contradiction probability is close to 1.0 for nearly any pair of opposing-stance arguments on the same topic, since an NLI model reads mere stance opposition as contradiction even when the two arguments do not actually engage the same underlying claim. This yields a stance-opposition graph rather than a rebuttal graph, and at that density PageRank centrality and multi-hop traversal would reflect little more than how many opposing arguments a policy happens to have, not genuine argumentative structure.

Since neither extreme, a near-universally connected stance-opposition graph nor an overly sparse one that discards genuine rebuttals, is usable, calibration targets a

middle band, informally set at roughly 5–15% saturation: wide enough to retain a meaningfully connected rebuttal structure per policy, but far below the 33% observed under the single-direction rule. Two changes were available to reach this target: replacing the single-direction acceptance rule with a *bidirectional* one, requiring the *minimum* of a pair’s forward and backward contradiction probabilities to clear a threshold τ_{contra} rather than either direction alone; and simply raising the cosine floor used to block candidate pairs before NLI is run at all. It was not obvious in advance which of the two would do the actual work of thinning the graph, so a dedicated calibration script was run on a small sample of policies, chosen to span the most-, median-, and least-saturated ends of the original single-direction graph, before committing to a full 71-policy regeneration. The script reblocks candidate pairs at a deliberately low cosine floor of 0.40, low enough to expose the full score distribution rather than one already truncated by the floor being calibrated, scores every candidate pair bidirectionally, and reports two diagnostics per pair beyond the raw forward and backward contradiction probabilities:

- ***contra_min***, the minimum of the forward and backward contradiction probabilities, $\min(P_{\text{fwd}}, P_{\text{bwd}})$: the quantity ultimately thresholded by the bidirectional acceptance rule.
- **Gap**, the absolute difference between the two directions, $|P_{\text{fwd}} - P_{\text{bwd}}|$: a diagnostic for whether bidirectionality is doing anything *beyond* what the blocking floor already does. If the gap is consistently close to zero, the two directions agree with each other almost everywhere, contradiction behaves as a symmetric property of the pair, and the bidirectional-minimum rule cannot thin the graph beyond what raising the cosine floor alone would already do. A gap that is often large would instead mean the two directions frequently disagree, and bidirectional-minimum is doing real, direction-sensitive filtering of its own.

On this corpus, contradiction probability turned out to be a close to binary quantity in each direction, clustering near 0 or 1 rather than spreading continuously between them, so the gap was small for the large majority of pairs: the cosine floor, not the bidirectional-minimum rule, is the dominant lever controlling saturation. The bidirectional rule was kept regardless of this finding, since it costs nothing to apply and still removes the minority of pairs where the two directions genuinely disagree.

With this evidence in hand, the calibration script sweeps a small grid of candidate operating points and reports the resulting saturation at each one:

- four candidate cosine floors: 0.50, 0.60, 0.70, 0.80;
- five candidate *contra_min* thresholds τ_{contra} : 0.60, 0.80, 0.90, 0.95, 0.99;
- for each of the resulting 20 (floor, threshold) pairs, the saturation, edges kept over all possible cross-stance pairs, in the calibration sample.

A floor of 0.70 together with $\tau_{\text{contra}} = 0.90$ was the combination selected, landing at approximately 11% saturation on the calibration sample, inside the targeted 5–15% band. Table 4.2 reports the effect of applying this operating point to the full 71-policy graph: the CONTRADICTS edge count falls from approximately 1.06 million (33% of possible cross-stance pairs, the single-direction rule) to 344,000 (9%, the bidirectional rule at this operating point), consistent with the saturation observed on the smaller calibration sample. This operating point, together with an analogous bidirectional threshold $\tau_{\text{entail}} = 0.60$ for EQUIVALENT edges, is used throughout the four-stage recovery pipeline described next; paraphrase relations do not exhibit the same saturation problem that motivated the calibration above, since two arguments rarely entail each other bidirectionally by accident, but the same bidirectional discipline is applied to them regardless.

Table 4.2: Effect of the bidirectional-minimum criterion on CONTRADICTS edge count, computed over the full 71-policy graph at a blocking floor of 0.70 and threshold of 0.90.

Acceptance rule	Contradicts edges	Saturation
Single-direction threshold	$\approx 1.06\text{M}$	33%
Bidirectional minimum	344,000	9%

4.1.3 Argument-Level Edge Recovery

Applying an NLI model to every pair of arguments within a policy is computationally wasteful, since the overwhelming majority of pairs are unrelated, so edge recovery is not a single classification step but a pipeline that narrows the full space of possible argument pairs down to a small, high-confidence edge set. With the operating point fixed by the calibration in Section 4.1.2, a blocking floor of 0.70 and a bidirectional threshold $\tau_{\text{contra}} = 0.90$ for CONTRADICTS, this proceeds in four stages:

1. **Blocking.** A cheap embedding-cosine filter, computed over SENTENCE-TRANSFORMERS/ALL[†] MPNET-BASE-V2 embeddings [reimers2019sbert, song2020mpnet], restricts the pairs that will ever be scored by the far more expensive NLI model. Cross-stance (PRO/CON) pairs with cosine similarity of at least 0.70 become candidates for CONTRADICTS; within-stance (PRO/PRO or CON/CON) pairs with cosine similarity of at least 0.75 become candidates for EQUIVALENT. This step alone discards the large majority of the $\binom{n}{2}$ possible pairs within a policy, since most arguments on a given policy are neither close paraphrases nor topically close enough to plausibly stand in either relation.
2. **Bidirectional NLI classification.** Every surviving candidate pair is scored twice by the CROSS-ENCODER/NLI-DEBERTA-V3-BASE cross-encoder [he2023debertav3], trained on SNLI and MultiNLI [bowman2015snli, williams2018mnli]: once with the first argument as premise and the second as hypothesis, and once with the roles reversed. NLI is a directional task, so a single scoring direction cannot

be assumed to represent the pair as a whole; scoring both directions is what makes the bidirectional acceptance rule below possible.

3. **Bidirectional thresholding.** A candidate is accepted as CONTRADICTS only if the *minimum* of its forward and backward contradiction probabilities clears $\tau_{\text{contra}} = 0.90$, and, symmetrically, as EQUIVALENT only if both directions of entailment probability clear $\tau_{\text{entail}} = 0.60$. An optional third-stage LLM adjudicator can additionally re-score pairs whose NLI confidence falls in an uncertain middle band, but this option was left disabled for the edges used throughout this thesis, so that every reported edge is decided by NLI alone.
4. **Conflict resolution.** Because CONTRADICTS and EQUIVALENT are each symmetric relations, an unordered pair is stored once rather than twice. A small number of pairs clear both the CONTRADICTS and the EQUIVALENT threshold simultaneously; for these, only the higher-confidence verdict is kept, so no argument pair is ever tagged with both relations at once.

4.1.4 PageRank Centrality

Once the CONTRADICTS edge set is finalized, PageRank [page1999pagerank] is computed over it to supply the centrality signal used by the graph-aware retrieval system (Section 4.3). PageRank was originally formulated for the directed graph of hyperlinks between web pages, where the score of a page p_i is defined recursively as

$$\text{PR}(p_i) = \frac{1-d}{N} + d \sum_{p_j \in M(p_i)} \frac{\text{PR}(p_j)}{L(p_j)}, \quad (4.1)$$

where N is the total number of nodes, $M(p_i)$ is the set of nodes linking to p_i , $L(p_j)$ is the out-degree of p_j , and $d \in [0, 1]$ is a damping factor that mixes the link-following term against a uniform teleport probability, $\frac{1-d}{N}$, of jumping to any node at random. In matrix form, writing $\mathbf{v}$ for the uniform teleport vector ($v_i = 1/N$) and $\mathbf{A}$ for the row-normalized transition matrix of the graph, this is the stationary distribution of

$$\mathbf{PR} = d \mathbf{A}^\top \mathbf{PR} + (1-d) \mathbf{v}, \quad (4.2)$$

solved in practice by power iteration rather than direct linear solution, since the graphs involved are far too large to invert directly.

Two adaptations are made to apply Equations 4.1–4.2 to the CONTRADICTS graph built in Section 4.1.3. First, because CONTRADICTS edges are symmetric by construction (an edge is accepted only if contradiction holds in both directions, Section 4.1.3), the graph is treated as undirected, so $M(p_i)$ and $L(p_j)$ in Equation 4.1 reduce to a node’s neighbors and degree in the symmetrized CONTRADICTS graph rather than to directed in-links and out-links. Second, alongside a single global PageRank computed with the standard uniform teleport vector $\mathbf{v}$, a *personalized* PageRank is computed separately for each policy, replacing $\mathbf{v}$ with a teleport vector $\mathbf{v}_p$ restricted to that policy’s own PRO and CON arguments,

$$v_{p,i} = \begin{cases} 1/|A_p| & \text{if } i \in A_p, \\ 0 & \text{otherwise,} \end{cases} \quad (4.3)$$

where A_p is the set of arguments attached to policy p by a SUPPORTS or ATTACKS edge. Restarting the random walk only at policy p 's own arguments measures centrality relative to that single policy's rebuttal structure, rather than to the corpus as a whole; an argument that is central within a small, tightly contested policy is not penalized for the fact that a much larger policy elsewhere in the corpus has more CONTRADICTS edges in absolute terms, since the global variant of Equation 4.1 would conflate the two. Both variants use a damping factor of $d = 0.85$, the standard choice inherited from the original formulation, and are computed by power iteration to an L1 residual tolerance of 10^{-10} or a maximum of 200 iterations, whichever comes first. It is the personalized, per-policy PageRank score, $\text{PR}_p(a) \triangleq \text{PR}(a; \mathbf{v}_p)$, that the graph-aware retrieval system in Section 4.3 uses as its relevance signal.

4.2 Retrieval Framework

Every retrieval system compared in this thesis, whether it uses the graph built in Section 4.1 or not, retrieves stance-aware: given a policy, it returns a top- k list of PRO arguments and a separate top- k list of CON arguments, so that a single policy query always yields two independent retrieved lists rather than one undifferentiated one. Both lists, for every system, are constructed the same way: by selecting k items from a candidate pool using Maximal Marginal Relevance (MMR) [9], introduced in Section 1.4.6 as the standard mechanism for balancing relevance against redundancy in a retrieved set. This section fixes the shared MMR mechanics that every system uses identically; what actually distinguishes one system from another, the candidate pool each one searches and the relevance signal each one ranks by, is deferred to Section 4.3.

4.2.1 Maximal Marginal Relevance

MMR builds a retrieved list greedily, one item at a time, rather than taking the k most relevant candidates outright: at each step it adds whichever remaining candidate maximizes a weighted combination of relevance to the query and dissimilarity to everything already selected. Writing R for the full candidate pool and $S \subseteq R$ for the set of items already selected (initially empty), the next item added to S is

$$\arg \max_{d_i \in R \setminus S} \left[\lambda \cdot \text{rel}(d_i) - (1 - \lambda) \max_{d_j \in S} \text{sim}(d_i, d_j) \right], \quad (4.4)$$

where $\text{rel}(d_i)$ is candidate d_i 's relevance score, defined per system in Section 4.3; $\text{sim}(d_i, d_j)$ is embedding cosine similarity between two candidates, computed over the same SENTENCE-TRANSFORMERS/ALL-MPNET-BASE-V2 embeddings [reimers2019sbert|song2020mpnet] used throughout this thesis; and $\max_{d_j \in S} \text{sim}(d_i, d_j)$ is therefore

the similarity between d_i and whichever already-selected item is closest to it, the redundancy penalty. The weight $\lambda \in [0, 1]$ controls the relevance/diversity tradeoff directly: $\lambda = 1$ reduces Equation 4.4 to plain top- k ranking by relevance alone, with no diversification, while $\lambda = 0$ ignores relevance entirely and greedily maximizes spread against what has already been picked. Every system and every experiment in this thesis fixes $\lambda = 0.5$, giving relevance and diversity equal weight; because this value never changes across systems, any diversity difference observed in the results (Chapter ??) is attributable to differences in the candidate pool R or the relevance signal $\text{rel}(\cdot)$ between systems, not to one system having been handed a more favorable relevance/diversity tradeoff than another.

The first item MMR selects is a useful special case for making Equation 4.4 concrete: with $S = \emptyset$, the redundancy term vanishes, and the first pick reduces to $\arg \max_{d_i \in R} \text{rel}(d_i)$, plain top-1 relevance ranking. Every subsequent pick then balances that same relevance signal against dissimilarity to whatever has been selected so far, so a system’s relevance signal alone determines where retrieval starts, and the interaction between that signal and the pool’s own similarity structure determines everywhere it goes after that.

Two design choices are deliberately left open by Equation 4.4, and it is exactly these two that distinguish one retrieval system from another in this thesis:

- how the candidate pool R is generated, that is, which arguments are even eligible to be retrieved before MMR is run at all; and
- what relevance signal $\text{rel}(\cdot)$ ranks the pool by.

Holding the corpus, the embedding model, the MMR mechanism, and k fixed across every system, and varying only these two choices, is what makes it possible to attribute an observed difference in retrieval quality to a specific design decision, candidate generation or relevance ranking, rather than to a confounded bundle of implementation differences. Section 4.3 defines each system compared in this thesis exactly by how it answers these two questions.

4.3 Retrieval Systems Compared

Four retrieval systems are compared throughout this thesis, all built on the shared MMR mechanism of Section 4.2 and differing in exactly one component at a time, the candidate pool R , the relevance signal $\text{rel}(\cdot)$, or the diversification step itself, holding the corpus, the embedding model, and k fixed across all four. This single-variable design is what licenses attributing an observed difference between two systems to the one component that actually differs between them, rather than to a confounded bundle of implementation choices.

4.3.1 StandardRAG: the Embedding-Only Baseline

StandardRAG is the embedding-only baseline against which the graph-aware system is compared, and uses no graph signal at any stage.

- **Candidate pool.** The pool R is the policy’s own native argument set, every argument already carrying a SUPPORTS or ATTACKS edge to the policy (Section 4.1), capped at 200 per stance. No query is embedded and no corpus-wide search is performed to build this pool: construction is a direct lookup of the policy’s own tagged arguments, truncated to 200 in the order they are stored rather than ranked by similarity to anything. Cosine similarity enters only at the relevance-signal stage below, not at candidate-generation time.
- **Relevance signal.** Once the pool is fixed, candidates are ranked by cosine similarity, over SENTENCE-TRANSFORMERS/ALL-MPNET-BASE-V2 embeddings [reimers2019sbert, song2020mpnet], to $\bar{d}$, the centroid of that same pool: $\text{rel}(d_i) = \text{sim}(d_i, \bar{d})$. This is a form of pseudo-relevance feedback, ranking candidates by centrality to an already-assembled candidate set rather than by similarity to an explicit query, and it is the only point at which embedding similarity plays any role in StandardRAG’s pipeline.

Because StandardRAG’s candidate pool is built directly from the SUPPORTS/ATTACKS edges already present in the graph (Section 4.1), rather than from a separate corpus-wide embedding search, it is graph-*adjacent* in its pool construction even though it consults no CONTRADICTS structure and computes no centrality score; “no graph signal” refers specifically to the relevance signal and to the absence of any multi-hop expansion, not to the origin of the pool itself. This matters for how StandardRAG compares to PR-GraphRAG below: under the native pool, the two systems’ candidate pools share the same direct-attachment seed, and differ only in whether that seed is expanded by one CONTRADICTS hop; the axis that separates them most consequentially under this condition is therefore the relevance signal, cosine-to-centroid versus personalized PageRank, with the pool itself diverging only at the margin, for policies whose native argument count for a stance exceeds the 200 cap.

4.3.2 PR-GraphRAG: the Graph-Aware System

PR-GraphRAG is this thesis’s proposed system, and is the only one of the four that consults the CONTRADICTS structure of the graph built in Section 4.1. Its candidate pool starts from the identical direct-attachment seed as StandardRAG’s (Section 4.3.1), so, under the native pool, the two systems are not distinguished by one having access to a wider or more permissive net over the corpus than the other; both are structurally confined to a single policy’s own tagged arguments at the outset. What PR-GraphRAG adds is a further expansion of that same seed along CONTRADICTS edges, and a relevance signal, personalized PageRank, that reflects the rebuttal structure connecting those arguments to one another rather

than their raw similarity to a pool centroid. The motivation this is built to address, an embedding-only signal conflating *an argument that addresses this policy* with *an argument that merely uses this policy’s vocabulary*, becomes directly relevant once the hardened, distractor-injected pools of Section 4.6 deliberately introduce arguments from topically adjacent sibling policies (for instance, several policies on cosmetic surgery, organ trade, and human cloning all sharing a Bioethics & Personal Autonomy MetaPolicy, Section 4.1.1) into a policy’s own candidate pool: at that point, an embedding-only relevance signal has no basis for distinguishing a genuine argument from a vocabulary-sharing distractor, whereas an argument’s position, or absence, in the CONTRADICTS structure surrounding the policy’s genuine arguments does.

- **Candidate pool.** The pool is seeded by the arguments holding a direct SUPPORTS or ATTACKS edge to the target policy, then expanded one hop along CONTRADICTS edges: a CON argument that contradicts a directly-attached PRO argument is added to the PRO candidate pool, and symmetrically, a PRO argument that contradicts a directly-attached CON argument is added to the CON pool. Because CONTRADICTS edges are recovered only within a policy (Section 4.1.3), this hop cannot itself reach outside the target policy on the native pool; its effect there is to recover same-policy arguments that the 200-item truncation of the direct seed may have dropped, not to widen the search to other policies. The hop’s exclusionary role, arguments with no path into the target policy’s own CONTRADICTS neighborhood are never added at all, becomes consequential once the pool itself is allowed to contain material from outside the policy, as it deliberately does under the hardened-pool construction of Section 4.6.
- **Relevance signal.** Candidates are ranked by their personalized, per-policy PageRank score (Section 4.1.4), min-max normalized within the pool, so $\text{rel}(d_i) = \widehat{\text{PR}}_p(d_i)$.

Under Equation 4.4, the first item PR-GraphRAG selects is therefore the most central argument in the policy’s rebuttal structure, the argument the largest weight of other arguments engages with, rather than the argument closest to a pool centroid.

4.3.3 RAG-no-MMR: a Diversification Ablation

RAG-no-MMR isolates the contribution of MMR’s diversification step in isolation from every other design choice. It uses the identical candidate pool and the identical relevance signal as StandardRAG (Section 4.3.1), the policy’s native argument set capped at 200, ranked by cosine similarity to the pool centroid, but selects its final top- k list by plain ranking on $\text{rel}(\cdot)$ alone rather than by running MMR (Equation 4.4) over the pool. Formally, this is the $\lambda = 1$ boundary case of Equation 4.4 applied specifically to StandardRAG’s pool and relevance signal, rather than a separately-tuned system. Because pool and relevance signal are held fixed and only the diversification step is

removed, any difference between StandardRAG and RAG-no-MMR in Chapter ?? isolates exactly what MMR’s redundancy penalty buys, or costs, on this corpus.

4.3.4 Random: a Sanity Floor

Random selects its top- k list uniformly at random from the same native, policy-restricted pool that StandardRAG and RAG-no-MMR draw from, using no relevance signal and no diversification step at all. It serves two purposes in this thesis’s evaluation: a sanity floor that any system using a genuine relevance signal ought to outperform, and, as Chapter ?? returns to, a diagnostic for whether a given evaluation pool is discriminative enough to separate a real retrieval system from one making no retrieval decision whatsoever.

4.3.5 Summary

Table 4.3 summarizes all four systems along the two axes fixed in Section 4.2, candidate pool and relevance signal, plus whether MMR diversification is applied. StandardRAG and RAG-no-MMR share an identical pool and relevance signal, differing only in the last column, isolating MMR’s contribution alone. StandardRAG and PR-GraphRAG share the same direct-attachment seed for their pool (both start from a policy’s own SUPPORTS/ATTACKS arguments) and differ in whether that seed is expanded by a CONTRADICTS hop and in which relevance signal ranks it, so that any observed difference between the two is attributable to the graph-derived expansion and ranking specifically, not to a pool built by an unrelated mechanism. This table is, in effect, a compact statement of the single-variable design that Chapter ?? exploits to attribute observed differences to specific components rather than to confounded implementation choices.

Table 4.3: The four retrieval systems compared in this thesis, defined along the two axes MMR leaves open (candidate pool, relevance signal) plus whether diversification is applied.

System	Candidate pool R	Relevance signal $\text{rel}(\cdot)$	MMR
StandardRAG	Native pool (SUPPORTS/ATTACKS seed), capped at 200	Cosine to pool cen- troid	Yes ($\lambda = 0.5$)
PR- GraphRAG	Same seed + 1-hop CON- TRADICTS expansion	Personalized PageRank	Yes ($\lambda = 0.5$)
RAG-no- MMR	Native pool (SUPPORTS/ATTACKS seed), capped at 200	Cosine to pool cen- troid	No (plain top- k)
Random	Native pool (SUPPORTS/ATTACKS seed), capped at 200	None (uniform)	No (ran- dom sample)

4.4 Use Cases

Retrieval quality is evaluated through two downstream use cases rather than one, chosen to reflect two genuinely different ways a moderator or citizen consumes retrieved arguments: scanning a short, high-precision list in full, and reading a longer synthesized account grounded in a wider set of retrieved material. The two use cases share every retrieval system, the corpus, and the candidate-pool construction described in Sections 4.1–4.3; they differ only in k , in whether the retrieved set is read directly or first summarized, and in the downstream metrics each one is scored on (Chapter ??).

4.4.1 UC1: Conflict-Map Retrieval

Given a policy, UC1 asks a retrieval system to return a short, stance-balanced set of PRO and CON arguments that a moderator can scan directly to see where a debate’s genuine points of disagreement lie, without having to read the corpus itself. This is a *precision-first* task: because the retrieved set is read in full rather than filtered further by a human or a downstream model, every irrelevant or redundant item carries a direct cost, and diversity, avoiding near-duplicate arguments that restate the same point, matters as much as raw relevance. UC1 is evaluated over the candidate pool of 200 fixed in Section 4.3, and reported primarily at $k = 5$, the operating point at which a human moderator would realistically read every retrieved item rather than skimming a long list; Chapter ?? additionally reports a sweep across pool sizes and k values to check whether conclusions drawn at this single operating point generalize.

4.4.2 UC2: Debate Summarization

Given a policy, UC2 asks a system to generate a short natural-language summary of the debate, covering the main PRO and CON positions in balance, grounded in that system’s retrieved arguments rather than in the corpus at large. Unlike UC1, this is a *recall-oriented*, generation-downstream task: retrieval quality matters here not because a person reads the retrieved list directly, but because it bounds what the summarizer can faithfully say about the debate. A summarizer cannot surface a position it was never given, so UC2 evaluates whether a retrieval system’s choices survive an additional generation step rather than being read off the retrieved list itself. UC2 is evaluated at $k = 15$, a wider retrieval budget than UC1’s, appropriate for feeding a summarization prompt with enough material to synthesize from rather than for direct human reading.

Summaries are generated with a structured prompt, issued to Qwen3:8b at temperature 0, that requires the model to return a JSON object with exactly five fields, rather than free-form prose: the strongest PRO argument, the strongest CON argument, the key tension between them stated as specific contradicting claims rather than a restatement of the policy topic, any missing perspectives, and an overall assessment of how balanced the retrieved debate is. Constraining the summarizer’s

output to this fixed schema keeps the four downstream UC2 metrics (Chapter ??) computable consistently across systems and policies, since every summary decomposes into the same five comparable fields regardless of which retrieval system fed it.

4.5 Policy Selection

Every result reported in this thesis is computed over a fixed sample of 30 policies out of the corpus’s 71, selected once and reused unchanged across every system, pool condition, and metric, so that no later comparison is confounded by different systems having been evaluated on different policies.

The obvious selection rule, simply taking the 30 policies with the most arguments, was rejected because it would bias the sample toward corpus outliers and foreclose any later analysis of how retrieval performance varies with graph density: Section 3.2 already established a roughly 1.7-fold spread in argument count across the corpus’s 71 policies (331 to 554), and a large-policies-only sample would systematically discard the sparse end of that range, the exact regime in which a graph-aware system’s one-hop expansion has the least material to work with. A tercile-stratified sample was used instead:

1. The 71 policies are sorted by total argument count and split into three bins of equal size, sparse, medium, and dense, 23, 23, and 25 policies respectively (71 does not divide evenly by three).
2. 10 policies are drawn uniformly at random from each bin, with a fixed random seed (seed = 42, the same seed used throughout this thesis’s other randomized steps, for consistency) for reproducibility, giving $10 \times 3 = 30$ policies in total.

This produces three evenly-sized groups spanning the corpus’s full density range by construction:

- **Dense tercile** (432–554 arguments per policy): 10 policies drawn from the 25 policies with the most native argument volume.
- **Medium tercile** (405–432 arguments per policy): 10 policies drawn from the 23 policies in the middle of the distribution.
- **Sparse tercile** (331–404 arguments per policy): 10 policies drawn from the 23 policies with the least native argument volume, the regime in which a graph-aware system’s one-hop CONTRADICTS expansion has the least material to work with.

Because the sample is balanced across these three groups by design, any relationship later observed between graph density and retrieval quality can be attributed to density itself rather than to an artifact of oversampling one density regime over another; this is what licenses the density-versus-performance analysis in Chapter ??.

4.6 Hardened Pool Construction

4.6.1 Why the Native Pool Is Not a Sufficient Test

Every candidate pool discussed so far, whether used by StandardRAG, PR-GraphRAG, RAG-no-MMR, or Random, is drawn from a policy’s own native argument set (Section 4.3). Inspecting the ground-truth relevance judgments over this native pool (the same judgments described in Section 4.7 below) reveals that it is, by construction, largely relevant already: averaged over the stratified 30-policy sample (Section 4.5), 57.7% of judged arguments in a native pool receive a relevance grade of 2 or higher, only 6.8% receive a grade of 0, and this relevance density varies almost not at all from one policy’s pool to another, a standard deviation of just 0.071 across the 30 sampled policies. The problem this creates is not specific to any one policy, it is systemic: because most arguments in a native pool are at least somewhat relevant simply by virtue of having been authored for that policy, a ranking metric has little room to separate a genuinely good retrieval system from a poor one, or, in the most extreme case, from Random (Section 4.3.4) itself. A relevance signal that does essentially nothing can still score respectably on a pool where most of what it could possibly return already clears the relevance bar.

This motivates deliberately constructing a *harder* evaluation pool rather than relying on the native one, following the broader argument-retrieval literature’s recognition that the specific distractors present in a retrieval context materially affect how discriminating an evaluation is, and that a retrieval evaluation should therefore construct adversarial candidate pools rather than rely on native, largely-relevant ones.¹ Concretely, the aim is to inject arguments that are topically adjacent to a policy, close enough in subject matter that an embedding-only signal cannot trivially reject them, but that do not actually address the policy in question, so that a system’s relevance signal is genuinely tested on its ability to discriminate rather than credited for a pool that is easy by construction.

4.6.2 Constructing the Distractor Pool

For a given target policy P and stance $s \in \{\text{PRO}, \text{CON}\}$, distractors are constructed in four steps:

1. **Identify sibling policies.** Siblings of P are the other policies sharing P ’s MetaPolicy (Section 4.1.1), for instance “We should legalize organ trade” and “We should ban human cloning” as siblings of “We should ban cosmetic surgery” under Bioethics & Personal Autonomy. Where a policy’s MetaPolicy grouping is unavailable, a fallback identifies siblings instead as the four policies whose centroid (Section 4.1.1) is nearest by cosine similarity to P ’s own centroid.

¹This methodological point follows Cuconasu et al.’s work on the effect of distractors in retrieval-augmented generation contexts (SIGIR 2024); a full citation for this source still needs to be added to the thesis bibliography.

2. **Gather candidates.** All same-stance arguments belonging to P 's sibling policies are pooled as candidate distractors: sibling PRO arguments feed P 's PRO distractor pool, sibling CON arguments feed P 's CON distractor pool. Any candidate duplicating, by text, an argument P already owns is discarded.
3. **Rank and select.** Candidates are ranked nearest-first by cosine similarity to P 's own centroid, the hardest and most topically confusable distractors, and the top D are kept for each injection level D (Section 4.6.3). A random-selection variant, drawing uniformly from the sibling candidate pool instead of nearest-first, is also available as a system-agnostic control, to check that any robustness result is not an artifact of the specific nearest-first selection strategy.
4. **Naturalize into the target policy.** Each selected distractor is not moved but *copied*: a fresh argument record is created with a new identifier, re-stamped to belong to P ($\text{policy} = P$) and to carry stance s , and flagged `is_distractor = true`, with the original policy and argument id retained as provenance fields. Re-stamping is what makes the distractor copy visible to every retrieval system on equal footing: every candidate-pool lookup in Section 4.3 keys off the target policy's own tagged arguments, so an argument that still carried its original policy label would never enter any system's candidate pool at all, defeating the purpose of injecting it.

An early version of this pipeline appended distractor copies to the end of each policy's native argument list, after the (up to 200) native arguments already occupying the first positions. Since every candidate-pool lookup in Section 4.3 truncates that list to 200, the appended distractors, sitting at position 200 and beyond, were silently sliced off before any system ever saw them: a sanity check that had the Random baseline (Section 4.3.4) retrieve zero distractors from a pool that was supposed to be roughly one-third distractors by construction, a result close enough to statistically impossible that it exposed the bug rather than confirming robustness. The fix interleaves the native and distractor identifiers with a deterministic, per-(policy, stance, level) seeded shuffle before any truncation happens, so a downstream $[:200]$ slice draws a representative native/distractor mixture at the intended injected rate. This interleaving step is applied only when distractors have actually been injected, so the zero-distractor control condition below remains byte-identical to the native pool.

4.6.3 Injection Levels

Distractors are injected at five levels per stance, $D \in \{0, 50, 100, 200, \text{full}\}$, where *full* takes every available sibling candidate rather than capping at a fixed count. $D = 0$ is a byte-identical control, reproducing the native pool exactly, since no injection and no interleaving shuffle occur at this level. The results reported in Chapter ?? focus primarily on $D = 100$ (denoted L100), chosen as a middle setting large enough to meaningfully stress-test a pool of up to 200 native arguments without the distractor

share overwhelming it entirely, with the full D sweep reported separately to check that conclusions drawn at L100 generalize across injection levels.

4.6.4 Four Edge-Connectivity Scenarios

Injecting distractor arguments raises a further question that is easy to overlook: once a distractor copy exists inside the target policy’s argument set, should it participate in the CONTRADICTS/EQUIVALENT edge-recovery pipeline of Section 4.1.3 the same way a native argument does? Answering this only one way would conflate two separate questions, whether graph-aware retrieval resists distractors at all, and specifically whether that resistance depends on distractors being graph-*isolated*. Four scenarios are therefore compared below, distinguished by whether, and how, distractor arguments are connected into the CONTRADICTS graph.

The first scenario, **no distractors**, is the native pool of Section 4.6.1, unmodified.

The second, **isolated distractors**, is the primary hardened condition used throughout Chapter ??: distractor copies are injected exactly as described in Section 4.6.2, but the edge-recovery pipeline of Section 4.1.3 is never re-run over them, so they carry no CONTRADICTS or EQUIVALENT edge to anything, native argument or fellow distractor alike. Under this scenario, a distractor can still enter a system’s candidate pool directly, since candidate-pool lookups key off the SUPPORTS/ATTACKS re-stamping from Section 4.6.2 rather than off CONTRADICTS connectivity, but it can never be reached by PR-GraphRAG’s one-hop expansion, and its personalized PageRank score is that of an isolated node.

The third and fourth scenarios both re-run the full three-stage edge-recovery pipeline of Section 4.1.3 over the merged native-plus-distractor argument set, at the same calibrated thresholds (Section 4.1.2), but differ in one blocking-stage rule. In **distractors with edges**, every pair is eligible for CONTRADICTS/EQUIVALENT scoring exactly as in the native pipeline, so two distractor copies, both drawn from sibling policies that may genuinely discuss related claims, can end up connected to each other as readily as two native arguments can. In **distractors, no distractor–distractor edges**, the blocking stage additionally skips any candidate pair where both endpoints are distractor copies, before NLI is ever run on that pair, while leaving native–distractor pairs to be scored normally; native–native pairs are unaffected in either scenario, so the two are directly comparable and differ only in whether distractors can connect to each other.

Table 4.4 summarizes the resulting connectivity of all four scenarios; Table 4.5 just below it reports the actual edge counts this produced at L100.

Run over the full merged native-plus-distractor argument set at L100, the unrestricted pipeline (**distractors with edges**) accepts 385,872 CONTRADICTS edges in total. Re-running the identical pipeline with the additional both-endpoints-distractor exclusion (**distractors, no distractor–distractor edges**) accepts 345,109 CONTRADICTS edges and 13,302 EQUIVALENT edges, with zero pairs triggering the Stage 4 conflict-resolution rule (Section 4.1.3) in this run. The 385,872-edge unrestricted

Table 4.4: The four pool-construction scenarios compared in this thesis, by whether injected distractor arguments participate in the CONTRADICTS graph.

Scenario	Distractor–distractor edges	Native–distractor edges
No distractors (native)	N/A	N/A
Isolated distractors	None	None
Distractors with edges	Unrestricted	Yes
No distractor–distractor edges	None	Yes

total decomposes into 344,556 native–native edges, matching the native-corpus count already established in Table 4.2; 553 native–distractor edges; and 40,763 distractor–distractor edges, the difference removed by the no-distractor–distractor-edges rule.

Table 4.5: Decomposition of the 385,872 CONTRADICTS edges generated over the merged native-plus-distractor argument set at L100, by pair type.

Pair type	Contradicts edges
Native–native	344,556
Native–distractor	553
Distractor–distractor	40,763
Total (unrestricted)	385,872

This breakdown is itself informative beyond justifying the four-scenario design: distractor arguments contradict *one another* at nearly two orders of magnitude the rate at which they contradict the target policy’s own genuine arguments, 40,763 distractor–distractor edges against only 553 native–distractor edges. Sibling-policy distractors, drawn from topically adjacent policies under the same MetaPolicy (Section 4.1.1), evidently argue with each other far more often than they engage the specific arguments already native to the target policy. This matters for interpreting the difference between scenarios 3 and 4 in Chapter ??: whatever connectivity the unrestricted scenario adds over the no-distractor–distractor-edges scenario is overwhelmingly internal to the injected distractor set itself, not a new pathway connecting distractors into the target policy’s own argumentative structure.

Of the 72,349 distractor–distractor candidate pairs that survived blocking but were excluded from NLI scoring entirely under the no-distractor–distractor-edges rule, roughly 56% (40,763 of 72,349) would have cleared the bidirectional CONTRADICTS threshold had they been scored, confirming that sibling-policy distractors frequently stand in a genuine contradiction relation to *each other*, exactly the connectivity that scenario 4 exists to exclude and that scenario 3 exists to test the consequences of. As a validation step before this edge set was used to recompute PageRank, the edge-count refresh script was run with the expected CONTRADICTS total passed explicitly as a guard (`-expected-contradicts 345109`), so any mismatch between the freshly generated edge file and the count reported by Stage 3 would halt the pipeline rather than silently propagate into the centrality scores PR-GraphRAG depends on.

Comparing all four scenarios makes it possible to attribute any observed robustness

specifically: whether graph-aware retrieval resists distractors at all (scenario 2 versus scenario 1), whether that resistance is contingent on distractors remaining graph-isolated (scenario 3 versus scenario 2), and, if resistance degrades under scenario 3, whether the cause is distractor–distractor connectivity specifically rather than distractor connectivity in general (scenario 4 as the isolating comparison against scenario 3).

4.7 Ground Truth Generation

Every metric reported in Chapters ??–?? that depends on a notion of relevance, most directly UC1’s nDCG, is only as trustworthy as the relevance judgments it is computed against. This section describes how those judgments were produced: an exhaustive, dual-annotator grading protocol applied to every candidate pool, native and hardened alike.

4.7.1 Grading Protocol

Each (policy, stance) candidate pool is graded on a four-point ordinal relevance scale, 0 (irrelevant) to 3 (essential), by two large language models from different model families, Llama 3.3 70B [grattafiori2024llama3] as the primary annotator and Gemma 31B [gemma2026gemma4] as the secondary. Two design choices in this protocol depart from a more convenient alternative, and both are deliberate rather than incidental:

- **The full pool is judged, not a shortlist.** Every candidate in a policy’s pool, up to the same 200-argument cap used by the retrieval systems themselves (Section 4.3), receives a grade, rather than only the first n arguments encountered or only the union of what the compared systems happen to retrieve. Judging a shortlist instead would silently make relevance a positional artifact, an argument sitting past the shortlist boundary can never be credited as relevant no matter how strong it is, and would under-judge exactly the tail region a system’s own retrieved list might reach into. Grading the full pool removes that bias, at the cost of a long tail of grade-0 judgments and a real risk of annotator drift across the several batches a 200-argument pool requires; the per-policy grade distribution is retained in the full grading report specifically so that such drift remains visible rather than silently absorbed into an aggregate statistic.
- **Two annotators, not one.** A single LLM annotator’s grades cannot be distinguished from that model’s idiosyncratic biases. Using two annotators from different model families, and reporting their agreement (Section 4.7.3), makes any such bias visible rather than silently baked into the ground truth. This follows established results showing that LLM assessors can approach trained human judges’ agreement on graded relevance judgments [thomas2024llmjudge, upadhyay2024umbrella, upadhyay2025largescale], while a single-annotator

design carries a documented risk of systematic, model-specific judging bias [panickssery2024selfpref].

4.7.2 Consensus Resolution

With two annotators graded independently, a single ground-truth grade per argument is derived by a documented consensus rule rather than by an arbitrary tie-break. The rule adopted is *minimum*: when the two annotators disagree, the lower of the two grades is kept, following the disagreement heuristic used by the TripJudge collection [althammer2022tripjudge], on the reasoning that relevance the two annotators could not agree on should not be treated as though it were confidently established. An argument’s relevance grade, the primary annotator’s grade, and the secondary annotator’s grade are all retained in the stored ground truth, so the consensus resolution is transparent and reversible rather than discarding the individual judgments once resolved. A binary relevant set, used wherever a metric needs a relevant/non-relevant cut rather than a graded score, is defined as grade ≥ 2 .

4.7.3 Inter-Annotator Agreement

Agreement between the two annotators is reported at two granularities, following FiRA [hofstatter2020fira] and TripJudge’s [althammer2022tripjudge] observation that agreement on the full ordinal scale and agreement on the binary relevant/non-relevant cut can diverge substantially, and that the binary cut is the granularity that actually determines the relevant set most metrics are computed against:

- **Ordinal agreement**, quadratic-weighted Cohen’s kappa [cohen1968kappa] over the full 0–3 grade scale, penalizing a disagreement of, for instance, grade 0 versus grade 3 far more than a disagreement of grade 1 versus grade 2. Computed per policy and averaged over the stratified 30-policy sample (Section 4.5), this reaches a mean of 0.584 (std 0.082, range 0.439–0.750 across policies), which the conventional bands of Landis and Koch [landis1977kappa] place in the *moderate* agreement range on average, with individual policies ranging as high as the *substantial* band.
- **Binary agreement**, unweighted Cohen’s kappa over the collapsed relevant (≥ 2) versus non-relevant (< 2) cut, reported separately since a metric like nDCG’s underlying relevant set is defined at this granularity rather than the full ordinal one. This reaches a mean of 0.528 (std 0.081, range 0.348–0.678), somewhat lower and more variable than the ordinal figure, consistent with a binary cut discarding some of the ordinal scale’s partial-credit structure.

Applying the annotation pipeline’s own operational thresholds (kappa ≥ 0.6 = HIGH, $0.4\text{--}0.6$ = MEDIUM, < 0.4 = LOW) to the ordinal figure, 13 of the 30 sampled policies fall in the HIGH band and the remaining 17 in MEDIUM, with none

falling into LOW. For 3 of the 30 policies (“We should subsidize vocational education,” “We should close Guantanamo Bay detention camp,” and “We should abandon the use of school uniform”), the secondary annotator’s grades were unavailable for that policy, so no kappa could be computed at all; the single available annotator’s grades were still used for the consensus grade in these cases, but they are excluded from the agreement statistics above rather than silently treated as perfect agreement.

One limitation of this agreement analysis should be stated plainly: it measures agreement *between two LLM annotators*, not agreement between an LLM annotator and a human judge. It is therefore an inter-model consistency check, evidence that the two annotators are not grading idiosyncratically relative to each other, rather than direct evidence that either annotator’s grades track genuine human relevance judgments. This thesis does not include a human-annotated validation sample to anchor the LLM grades against; doing so would strengthen the ground truth’s claim to validity beyond inter-annotator consistency.

4.7.4 Grading the Hardened Pools

The native ground truth is not re-generated for the hardened pools of Section 4.6. Instead, grading proceeds incrementally: every native argument’s grade is carried over unchanged from Section 4.7.1, and only the injected distractor arguments, which by definition were never judged against the target policy, are graded fresh, by the identical dual-annotator, MIN-consensus protocol used for native arguments. Reusing native grades verbatim rather than re-running the annotators over them is deliberate: because LLM annotators are not perfectly deterministic across calls, re-grading the native arguments would perturb their existing labels for no benefit and would break the zero-distractor condition’s status as a byte-identical control (Section 4.6.3).

A distractor argument is graded on its merits rather than assumed irrelevant by construction. Although a distractor is, by design, an argument that does not belong to the target policy (Section 4.6.2), a sibling-policy argument drawn for its topical closeness to the target is not guaranteed to be irrelevant to it: stamping every distractor grade 0 outright would not measure the hardened pool’s actual relevance composition, it would simply assert it. Grading distractors independently is therefore what makes it possible to report, rather than assume, what fraction of an injected distractor pool a system could legitimately have retrieved.

What the L100 grading report does establish directly is whether distractor injection achieved its purpose: making the evaluation pool harder, in the specific sense of lowering its relevance density below the 57.7% figure that motivated hardening in the first place (Section 4.6.1). Computed over the stratified 30-policy sample at $D = 100$, the merged native-plus-distractor pool’s realized relevance base rate, 6,835 relevant arguments (grade ≥ 2) out of 17,820 graded pool entries, is 38.4%, a substantial drop from the native pool’s 57.7%. This is not uniform across policies: per-policy base rates in this sample range from 28.5% (“We should prohibit women in combat”) to 49.4% (“Homeschooling should be banned”), but every one of the

30 sampled policies falls below the native pool's 57.7% figure, confirming that the hardening construction lowers relevance density consistently across the sample rather than only on average.

Chapter 5

Evaluation

Chapter 4 fixed everything that produces a retrieved list or a generated summary in this thesis: the four retrieval systems, the shared MMR mechanism they all select through, the two use cases and the operating points each is evaluated at, the hardened-pool construction that stresses retrieval under adversarial distractor pressure, and the dual-annotator protocol that turns every candidate argument into a graded relevance judgment. What remains is to turn a retrieved list, or a summary generated from one, into the numbers Chapter ?? reports and compares across systems. That is the subject of this chapter.

The two use cases give rise to two largely disjoint metric families, because they ask a retrieval system to produce two different kinds of output. UC1’s retrieved lists are read directly, so its metrics ask the questions a moderator scanning the list would ask: are the most relevant arguments near the top, is the set free of near-duplicates, does it contain genuine pro/con tension rather than merely topic-adjacent material, and, once the pool is deliberately hardened, how much of what came back is contamination by design. UC2’s output is a generated summary rather than a list, so its metrics ask a different family of questions: does the summary say what the retrieved arguments actually said, does it invent anything the retrieved arguments do not support, and does it represent both sides of the debate fairly. Section ?? defines the first family and Section ?? the second.

Within each family, one subset of metrics forms the primary comparison this thesis reports across systems, matching the metric set reported in the conference paper this thesis expands on. A second subset is retained as supplementary diagnostics: metrics computed throughout evaluation-protocol development that surfaced a property of the metrics themselves worth documenting in its own right (most notably in Section ??), or that this thesis develops in more depth than the original paper did. Both subsets are defined here in full, with the primary or supplementary status of each metric stated explicitly rather than left for the reader to infer from which table it eventually appears in.

Before either family can be computed, every retrieved argument has to be resolved to a ground-truth grade. Section ?? defines that resolution step first, since nearly every metric that follows in Section ?? depends on it.

5.1 Resolving Retrieved Arguments to Ground-Truth Grades

Every UC1 relevance metric, and UC2’s Coverage metric, needs a way to look up a retrieved argument’s grade in the dual-annotator ground truth of Section 4.7. For a system whose candidate pool is drawn directly from a policy’s own graded arguments, this lookup is close to trivial: the retrieved text is one of the exact strings the annotators graded, so an exact, whitespace-stripped string match against the ground truth resolves the large majority of retrievals for StandardRAG, RAG-no-MMR, and Random. PR-GraphRAG is the case that requires more care. Its one-hop CONTRADICTION expansion (Section 4.3.2) can, under the hardened pool, pull in a distractor argument that was graded as part of the distractor manifest rather than the target policy’s own pool (Section 4.7.4), and, more rarely under the native pool, a genuine same-policy argument that the 200-item truncation happened to exclude from what was graded. An exact match against the target policy’s own graded pool then fails, not because the argument is unjudged, but because it was judged under a different key.

A retrieved argument that fails exact match is resolved by falling back to semantic matching: it is embedded with the same SENTENCE-TRANSFORMERS/ALL-MPNET-BASE-V2 model used throughout this thesis [reimers2019sbert, song2020mpnet], compared by cosine similarity against every argument the retrieving system had graded access to, and assigned the grade of its single closest match if that similarity clears $\tau_{\text{match}} = 0.75$, or treated as unmatched (grade 0, no credit) otherwise. An earlier version of this evaluation pipeline skipped the fallback entirely and resolved every retrieval by a plain substring lookup, so any retrieved argument phrased as even a close paraphrase of a graded one, of which the corpus has many, silently scored as a complete miss regardless of how relevant it actually was. This inflated the apparent gap between systems for reasons that had nothing to do with retrieval quality, since some systems happened to retrieve more verbatim pool members than others rather than because their retrieved arguments were less relevant, and it was diagnosed and corrected before any of the results in Chapter ?? were produced. Two design choices in the resulting fallback matter for what the metrics defined later in this chapter actually measure.

- **Exact match is tried first, semantic match only as a fallback.** Every argument in the judged pool is looked up verbatim before any embedding is computed, so resolution for the three systems that never leave the judged pool is exact and introduces no matching noise at all. The embedding-based fallback exists specifically for the minority of retrievals that can fall outside the pool a policy’s own arguments were graded from, and is applied only to those, rather than replacing exact lookup everywhere and paying its similarity-threshold uncertainty on every retrieval regardless of whether it is needed.
- **Resolution is single-best-match, not nearest-any-relevant-item.** A

retrieved argument is credited with the grade of the one ground-truth argument it is closest to, not the highest grade found among any ground-truth argument within the similarity threshold. This closes what would otherwise be a saturation loophole on self-similar pools: without single-best-match resolution, a retrieved argument sitting near a cluster containing one highly-graded item could inherit that item’s grade even if the retrieved argument itself is a weaker, less specific restatement of the same point, inflating every relevance metric that depends on the resolved grade.

Every relevance-dependent metric defined in Section ?? is written in terms of the grade this resolution step assigns, written $g(d)$ for a retrieved argument d , together with a match type, exact, semantic, or unmatched, that this thesis reports as a diagnostic alongside the relevance metrics themselves: a system whose top- k list resolves mostly by exact match is retrieving verbatim pool members, while a rising semantic-match fraction signals that a growing share of its retrievals are reaching outside the graded pool, which is expected of PR-GraphRAG under the hardened pool and is otherwise a signal worth inspecting on its own.

5.2 UC1 Metrics: Conflict-Map Retrieval

Table ?? summarizes the eleven quantities computed over every UC1 retrieved list. Five of them, nDCG@ k , MeanRel@ k , Conflict density, EILD, and Distractor-rate@ k , are the primary metrics this thesis reports as its headline UC1 comparison across systems, matching the metric set reported in the underlying conference paper. The remaining six, Precision@ k , Recall@ k , S-Recall@ k , raw ILD, the LLM rubric judge, and the abandoned pairwise judge, are supplementary: computed throughout evaluation-protocol development, retained in the full evaluation report, and used in this chapter and in Chapter ?? either as interpretive diagnostics or, in the case of raw ILD, specifically to expose a property of the primary metric it was replaced by. This design continues a tradition of graded, rank-aware relevance evaluation for argument retrieval established by shared evaluation tasks in the area [touche2022, perspectivearg2024], adapted here to a setting where every candidate pool, not just a system’s top- k output, has been exhaustively graded (Section 4.7.1).

5.2.1 Primary Relevance Metrics: nDCG@ k and MeanRel@ k

The primary relevance metric is graded nDCG@ k [jarvelin2002ndcg], computed with exponential rather than linear gain,

$$\text{DCG@}k = \sum_{i=1}^k \frac{2^{g(d_i)} - 1}{\log_2(i + 1)}, \quad \text{nDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}, \quad (5.1)$$

where $g(d_i) \in \{0, 1, 2, 3\}$ is the resolved grade of the argument at rank i (Section ??) and IDCG@ k is the DCG of the ideal ranking, the k highest consensus grades available in the judged pool, sorted descending. Exponential gain rewards placing

Table 5.1: UC1 metrics at a glance. Status marks whether a metric is part of this thesis’s primary reported comparison or a supplementary diagnostic. Pool marks whether a metric is meaningful under the native pool, the hardened pool, or both.

Metric	Status	Range	Pool
nDCG@ k	Primary	[0, 1]	Native & hardened
MeanRel@ k	Primary	[0, 3]	Native & hardened
Conflict density	Primary	[0, 1]	Native & hardened
EILD	Primary	[0, 1]	Native & hardened
Distractor-rate@ k	Primary	[0, 1]	Hardened only
Precision@ k	Supplementary	[0, 1]	Native & hardened
Recall@ k	Supplementary	[0, 1]	Native & hardened
S-Recall@ k	Supplementary	[0, 1]	Native & hardened
ILD (raw)	Supplementary	[0, 1]	Native & hardened
LLM rubric judge (3 axes)	Supplementary	[1, 5]	Native & hardened
Pairwise judge	Abandoned	–	–

a grade-3 argument above a grade-2 one far more heavily than linear gain would, which matches how UC1 is actually read: a moderator scanning a five-item conflict map benefits far more from having the single most essential argument at the top than from a list that is merely, on average, somewhat more relevant. Because the native and hardened pools are both graded exhaustively rather than only over a positional shortlist (Section 4.7.1), IDCG@ k is well defined for every policy and stance, an argument that would belong in the ideal top- k can never be missing from the denominator simply because it fell past wherever a shortlist boundary happened to be drawn.

nDCG@ k is reported at both operating points fixed in Chapter 4, $k = 5$ under the native pool and $k = 5$ under the hardened pool, and Chapter ?? additionally reports it across the pool-size and k sweep introduced there to check whether conclusions drawn at the single $k = 5$ operating point generalize. A linear-gain variant, DCG@ $k = \sum_i g(d_i) / \log_2(i + 1)$, is retained as a robustness check rather than reported as a second primary metric: it exists to confirm that any ordering observed under exponential gain is not an artifact of the specific gain function chosen, not because it measures something the exponential variant does not already capture.

MeanRel@ k is nDCG@ k ’s rank-free companion,

$$\text{MeanRel@}k = \frac{1}{k} \sum_{i=1}^k g(d_i), \quad (5.2)$$

the mean resolved grade of the top- k on the original 0–3 scale, dividing always by k rather than by the number of items actually returned, so a system cannot raise its score simply by returning fewer arguments than requested. MeanRel@ k sits between a binary relevance metric, which collapses the full ordinal grade scale to a single relevant/non-relevant cut, and nDCG@ k , which uses the full grade scale but folds it together with rank position. Reporting it alongside nDCG@ k answers a question nDCG@ k alone cannot: whether an observed difference between two systems

reflects a genuine difference in the grades their retrieved arguments carry, or only a difference in how those grades happen to be ordered within the list. Two systems that retrieve the identical multiset of grades in different orders receive different $\text{nDCG}@k$ scores but the same $\text{MeanRel}@k$, so a gap that appears in $\text{nDCG}@k$ but closes in $\text{MeanRel}@k$ is a ranking difference, while a gap that survives in both is a difference in what was retrieved, not merely how it was arranged.

5.2.2 Rank-Free Diagnostics: Precision@k, Recall@k, and Subtopic Recall

Three further relevance-based quantities are computed as supplementary diagnostics, each answering a narrower question than $\text{nDCG}@k$ or $\text{MeanRel}@k$ and each retained for a specific interpretive reason rather than as redundant coverage of the same construct.

Precision@k uses the binary relevant/non-relevant cut already fixed in Section 4.7.2, grade ≥ 2 ,

$$\text{Precision}@k = \frac{1}{k} |\{i \leq k : g(d_i) \geq 2\}|, \quad (5.3)$$

resolved, like every relevance metric in this chapter, by single-best-match (Section ??): a retrieved item counts as a hit only if its own resolved grade clears the threshold, not because it happens to sit near some other relevant item in embedding space. Precision@k is easier to read at a glance than a graded metric, and, because it depends only on the binary cut rather than the full grade scale, it is the metric most directly comparable to the Distractor-rate@k metric of Section ??, both being simple fractions of the top-k that satisfy a yes/no condition.

Recall@k is the fraction of the judged pool’s relevant arguments (grade ≥ 2 , deduplicated by ground-truth text) that appear anywhere in the top-k,

$$\text{Recall}@k = \frac{|\{a \in \text{pool} : g(a) \geq 2\} \cap \{d_1, \dots, d_k\}|}{|\{a \in \text{pool} : g(a) \geq 2\}|}. \quad (5.4)$$

Full-pool judging, rather than judging only a shortlist or only the union of what compared systems retrieve, is what makes this denominator well defined at all, following the same incomplete-judgment concern documented for pooled retrieval evaluation more generally [buckley2004bpref]: a fixed, judged relevant set that every system is measured against, rather than one collected only from what was retrieved, avoids penalizing a system for surfacing a legitimate relevant argument that a narrower judging protocol never saw. Recall@k is retained for continuity with earlier evaluation passes, but this thesis reports it as a demoted diagnostic rather than a primary metric, because its denominator is structurally uninformative at the operating points UC1 is actually reported at. The native pool’s relevance base rate is 57.7% (Section 4.7.3), so a typical 200-item pool contains on the order of 115 relevant arguments per stance, meaning even a perfect top-5 retrieval can never clear roughly $5/115 \approx 0.04$ Recall@5, and the hardened pool’s lower 38.4% base rate (Section 4.7.4)

still leaves a denominator in the same range. Recall@ k at $k = 5$ or $k = 15$ is therefore compressed into a narrow band near zero for every system regardless of retrieval quality, and any small differences that do appear are dominated by noise in which few relevant items happen to fall in the top- k rather than by a meaningful difference in coverage.

S-Recall@ k is this thesis’s response to that problem, following the subtopic-retrieval framing of **zhai2003srecall**: rather than asking what fraction of individually relevant arguments a system surfaces, it asks what fraction of the distinct *points* present in the relevant set a system surfaces at least one instance of. The ground truth stores only a (text, grade) pair per argument, with no subtopic label, so a notion of subtopic has to be operationalized before S-Recall@ k can be computed at all. This thesis operationalizes it by clustering the judged pool’s relevant arguments, per policy and stance, with agglomerative average-linkage clustering over cosine distance between SENTENCE-TRANSFORMERS/ALL-MPNET-BASE-V2 embeddings, at a distance threshold of 0.15 (cosine similarity ≈ 0.85) and no fixed number of clusters, since how many distinct argumentative points a given policy’s relevant set actually contains is not known in advance and should vary from one policy to the next. S-Recall@ k is then the fraction of the resulting clusters represented at least once, by a relevant, resolved retrieval, in the top- k :

$$\text{S-Recall@}k = \frac{|\{c(a) : a \in \{d_1, \dots, d_k\}, g(a) \geq 2\}|}{n_{\text{subtopics}}}, \quad (5.5)$$

where $c(a)$ denotes the subtopic cluster a relevant retrieved argument a resolves to and $n_{\text{subtopics}}$ is the total number of clusters found for that policy and stance. Clustering the relevant set into subtopics this way is a defensible operationalization rather than the only possible one, and the specific distance threshold is a genuine methodological choice rather than a value with an independent justification of its own. A sensitivity check across two or three neighboring thresholds would strengthen the claim that S-Recall@ k ’s conclusions are not an artifact of exactly where that threshold was set, and is noted here as a limitation of the current operationalization rather than carried out in this thesis.

5.2.3 Diversity: From Intra-List Distance to Relevance-Constrained Diversity

Diversity is not an afterthought in UC1’s evaluation design, it is the second half of exactly the tradeoff MMR is built to manage (Section 4.2.1), so a metric that measures it directly is needed alongside the relevance metrics of Sections ??–??. Diversity as a first-class retrieval objective has a long history in recommendation and IR evaluation [**ziegler2005diversification**, **clarke2008novelty**] and continues to be treated as one in recent retrieval-augmented generation work [**lu2026diversity**]. The most direct, reference-free way to measure it is intra-list distance,

$$\text{ILD} = 1 - \text{mean}_{i < j} \text{sim}(e_i, e_j), \quad (5.6)$$

one minus the mean pairwise cosine similarity over every pair of items in the retrieved set, following Carbonell and Goldstein [9]. ILD is exactly the quantity MMR’s redundancy term (Equation 4.4) greedily works against at selection time, so it is the natural first choice for measuring what that term bought a system, and this thesis reports it as a supplementary diagnostic for that reason.

Raw ILD, however, breaks in a specific and instructive way once the pool is hardened. Injected distractor arguments are drawn from topically adjacent sibling policies (Section 4.6.2), close enough to the target policy to be plausible near-misses, but they are not close to the target policy’s own genuine arguments in embedding space in the way two genuine arguments on the same specific point tend to be. A system that retrieves several such distractors alongside a few genuine arguments therefore scatters its retrieved set widely across embedding space almost by construction, since the distractors sit far from the genuine cluster and often far from each other as well, and raw ILD rewards exactly this scattering regardless of whether any of it is relevant. A retrieval system could, in principle, score near the top of the ILD distribution by retrieving five mutually unrelated, entirely irrelevant arguments, which is the opposite of what UC1’s diversity criterion is meant to reward: a conflict map that spans several genuinely distinct points of contention, not one that merely spans several unrelated topics.

EILD, an expected, relevance-constrained variant of intra-list distance, is the primary diversity metric this thesis reports for exactly this reason. It restricts the pairwise computation of Equation ?? to the subset of the retrieved list that resolves to a relevant grade,

$$\text{EILD} = 1 - \text{mean}_{\substack{i < j \\ g(d_i) \geq 2, g(d_j) \geq 2}} \text{sim}(e_i, e_j), \quad (5.7)$$

returning 0 when fewer than two relevant items are retrieved, which occurs occasionally at $k = 5$ and becomes rare at the wider $k = 15$ operating point UC2 draws from. A grade-0 distractor is excluded from every pair EILD sums over, so a retrieval set padded with irrelevant, embedding- distant filler can no longer inflate its diversity score the way it can inflate raw ILD. What EILD measures instead is genuinely the construct UC1 cares about: among the arguments a moderator would actually credit as relevant, how varied are the distinct points they make. This thresholded construction follows the broader relevance-aware turn in diversity and novelty evaluation [ziegler2005diversification, clarke2008novelty], restricting the diversity computation itself to a relevant subset rather than treating diversity and relevance as two independent criteria, though it stops short of a fully relevance-weighted formulation in which every pair’s contribution would be scaled continuously by both items’ grades rather than gated by a single threshold. This thesis reports raw ILD alongside EILD, labeled explicitly as a diagnostic, specifically so that the artifact just described

remains visible in the full evaluation report rather than silently disappearing once EILD replaced it as the headline diversity number.

Concretely, suppose a $k = 5$ retrieved set resolves to grades 3, 2, 0, 2, 0. Raw ILD averages over all $\binom{5}{2} = 10$ pairs, including every pair that touches one of the two grade-0 items, so a system that happened to retrieve two embedding-distant distractors at ranks 3 and 5 sees its ILD pulled upward by six of those ten pairs regardless of how similar the three genuinely relevant items are to one another. EILD averages only over the $\binom{3}{2} = 3$ pairs formed by the grade-3 and two grade-2 items, so if those three genuine arguments happen to restate a similar point, EILD reports that redundancy directly instead of letting the two irrelevant items mask it.

5.2.4 Distractor-Rate@ k

Distractor-rate@ k is a direct provenance metric, meaningful only under the hardened pool, that counts the fraction of the top- k originating from the injected distractor manifest (Section 4.6.2) rather than from the target policy’s own native argument set,

$$\text{DR@}k = \frac{|\{d_1, \dots, d_k\} \cap \text{distractor manifest}|}{k}. \quad (5.8)$$

Unlike every relevance metric in this chapter, Distractor-rate@ k needs no resolved grade at all: it asks a question about where a retrieved argument came from, not about how relevant it is, and the distractor manifest recorded at injection time (Section 4.6.2) answers that question directly. This makes Distractor-rate@ k an independent cross-check on the grade-based relevance metrics rather than a restatement of them in different units. A distractor is not, by construction, guaranteed to be irrelevant to the target policy, a sibling-policy argument drawn for topical closeness can occasionally be graded relevant on its own merits (Section 4.7.4), so a low Distractor-rate@ k and a high nDCG@ k are correlated but not identical claims: the first says a system’s structural filtering kept contamination out of the retrieved list, the second says what it did retrieve was well graded, and a system could in principle score well on one without the other.

5.2.5 Conflict Density

Every metric defined so far asks whether a retrieved argument is relevant to its policy. Conflict density asks a different question entirely, one that goes directly to what UC1 is actually for: does the retrieved PRO set engage the retrieved CON set, or does it merely sit alongside it. A retrieved list can be highly relevant by every grade-based metric above while still failing UC1’s purpose if its pro and con arguments talk past each other rather than contesting the same underlying claims, since a conflict map that does not show genuine conflict has not done its job regardless of how relevant each individual argument is. Conflict density is the mean natural-language-inference contradiction probability over every retrieved PRO×CON pair,

$$\text{ConfDensity} = \frac{\text{mean}}{p \in \text{PRO}, c \in \text{CON}} P(\text{contradict} \mid p, c), \quad (5.9)$$

computed by the identical CROSS-ENCODER/NLI-DEBERTA-V3-BASE classifier [he2023debertav3] introduced in Section 4.1.3 for CONTRADICTS edge recovery, applied here to retrieved argument pairs rather than to candidate corpus pairs. The two uses of this NLI classifier differ in what they do with its output. Edge recovery converts a contradiction probability into a binary accept/reject decision, requiring the bidirectional minimum of forward and backward scores to clear $\tau_{\text{contra}} = 0.90$ (Section 4.1.2), because a graph edge is either present or absent. Conflict density instead reports the continuous mean probability directly, with no threshold and no bidirectionality requirement, because it is a diagnostic score meant to be averaged and compared across systems rather than a structural decision that has to be made once and stored. A retrieved set can therefore register a moderate conflict density even where no individual pair would have cleared the edge-recovery threshold, which is expected: conflict density is measuring the degree of argumentative tension present in what was retrieved, not counting how many CONTRADICTS edges happen to already exist between those specific arguments in the graph.

5.2.6 LLM Rubric Judging and the Abandoned Pairwise Comparison

The five metrics of Sections ??–?? are all decomposed, verifiable quantities: each one reduces to a well-defined computation over resolved grades, embeddings, or NLI scores, with a narrow and inspectable failure mode. This thesis also computes a holistic quality judgment as a supplementary cross-check, following the LLM-as-judge line of evaluation [zheng2023judging, liu2023geval], on the reasoning that a decomposed metric suite can miss a failure only a reader of the full retrieved set would notice, and that agreement between the holistic judgment and the decomposed metrics is itself evidence that the decomposed metrics are measuring what they claim to.

The first design tried for this holistic judgment was a binary, double-sided pairwise comparison: an LLM judge shown two systems’ retrieved sets for the same policy and asked which one it preferred, each pair judged in both presentation orders to cancel positional bias, a precaution motivated by documented evidence that LLM judges are not immune to such artifacts [wang2024fair, panickssery2024selfpref]. This design was tried and abandoned. Because the systems compared in this thesis retrieve from overlapping, often near-identical candidate pools under the native condition, a forced binary choice produced tie rates in the 73–97% range across policies, leaving too little discriminative signal to be useful for anything beyond confirming that the judge could not reliably tell two similar retrieved sets apart.

The rubric judge that replaced it scores each system’s retrieved set independently rather than comparatively, on a 1–5 scale across three named axes, using Llama 3.3 70B [grattafiori2024llama3] at temperature 0 for reproducibility:

- **Conflict engagement.** Do the PRO and CON arguments directly engage each other, attacking the same premises and dimensions, rather than talking past each other, the same construct Conflict density (Section ??) targets through NLI rather than holistic judgment.
- **Redundancy avoidance.** Does each argument in the set add a distinct point, with little repetition within it, the same construct EILD (Section ??) targets through embedding distance rather than holistic judgment.
- **Balanced writing aid.** Would this retrieved set equip a citizen to write a balanced, well-informed position paper covering the strongest pros, the strongest cons, and the underlying value tensions between them, a construct none of the decomposed metrics above measures directly.

Scoring each set independently rather than pairwise restores meaningful spread, since ties then occur only where two sets are judged genuinely equal on an axis rather than being forced whenever a comparative judgment is too close to call. The rubric judge is reported as a supplementary cross-check rather than promoted into the primary metric set, both because it lacks the narrow, verifiable failure mode of the decomposed metrics above and because its usefulness depends on whether it actually separates systems, a question Section ?? returns to directly for UC2’s version of the same judge, where the answer turns out to be less favorable than it is here.

5.3 UC2 Metrics: Debate Summarization

UC1’s metrics all operate on a retrieved list. UC2 asks a different question of the same underlying retrieval: once a system’s top- k arguments are handed to a summarizer, does the resulting summary preserve what made that retrieval good or bad, or does the generation step wash those differences out. This question is not answerable from UC1’s metrics alone, since retrieval quality and downstream generation quality are related but distinct constructs that need not move together [samuel2026beyond], which is exactly why this thesis evaluates both use cases rather than treating UC1 as a sufficient proxy for UC2. Summaries are generated with Qwen3:8b [yang2025qwen3] at temperature 0, from the structured five-field prompt fixed in Section 4.4.2, and this section defines the four automatic metrics computed over the resulting summary text.

5.3.1 Coverage

Coverage measures whether the summary omits genuine debate content that was available to it, operationalized as NLI-entailment-based recall of the retrieved argument set,

$$\text{Coverage} = \frac{|\{a \in \text{retrieved} : \max_{s \in \text{summary}} P(\text{entail} \mid s, a) > 0.5\}|}{|\text{retrieved}|}, \quad (5.10)$$

the fraction of retrieved arguments for which some sentence in the summary entails it. Framing coverage as entailment rather than lexical overlap follows the NLI-based line of summarization evaluation [laban2022summac], and specifically the recall-oriented framing used in general summarization evaluation [fabri2021summeval] and in retrieval-augmented generation evaluation toolkits [es2024ragas], adapted here to the retrieved-argument set rather than to a reference summary: an argument the summary paraphrases or restates counts as covered even where no words overlap, while an argument the summary never mentions, regardless of how it is worded, does not. The entailment probability is computed by the same CROSS-ENCODER/NLI-DEBERTA-V3-BASE classifier used throughout this thesis (Section 4.1.3, Section ??), applied here to summary-sentence and source-argument pairs rather than to argument-argument pairs, and the 0.5 threshold is the same entailment cutoff Section ?? reuses for Stance Balance below.

5.3.2 Faithfulness

Faithfulness measures the opposite failure mode: does the summary assert anything not grounded in what was actually retrieved. Rather than a holistic judgment of factual accuracy, Faithfulness is computed as structured, per-claim verification, following the claim-decomposition approach to generation evaluation [liu2023geval, es2024ragas]. An LLM judge, Llama 3.3 70B at temperature 0, first extracts every distinct factual or argumentative claim from the summary text, explicitly excluding mere framing statements that assert nothing substantive about the debate, and then decides for each extracted claim whether it is supported, directly stated or clearly entailed, by the retrieved source arguments the summarizer was given. Faithfulness is the resulting fraction,

$$\text{Faithfulness} = \frac{\# \text{ supported claims}}{\# \text{ claims in summary}}. \quad (5.11)$$

This binary, per-claim outcome is what distinguishes Faithfulness from a holistic rubric score of the kind Section ?? defined for UC1: each claim receives an independently checkable supported/unsupported verdict against a specific source passage rather than a single scalar impression of the summary as a whole, closer in kind to Coverage and Precision@ k than to the rubric judge, which is exactly why it is reported among UC2’s primary four metrics rather than alongside the rubric judge Section ?? excludes.

5.3.3 Stance Balance

UC2 was introduced in Section 4.4.2 as helping a citizen understand both sides of a debate, not only whichever side a system happened to retrieve more of, so a metric that measures whether the generated summary actually reflects both sides is needed directly rather than inferred from Coverage or Faithfulness alone. Stance Balance reuses the same NLI entailment machinery as Coverage, applied separately to the retrieved PRO arguments and the retrieved CON arguments: for each stance, it

counts how many of that stance’s retrieved arguments the summary entails above the same 0.5 threshold, and computes the proportion of covered content attributable to the PRO side,

$$\text{pro_frac} = \frac{\text{covered}_{\text{pro}}}{\text{covered}_{\text{pro}} + \text{covered}_{\text{con}}}, \quad \text{StanceBalance} = 1 - 2 |\text{pro_frac} - 0.5|. \quad (5.12)$$

A summary that entails an equal number of PRO and CON arguments scores 1.0, perfect balance, and a summary that entails arguments from only one side scores 0.0, regardless of how many arguments from that one side it covers. Stance Balance is deliberately independent of Coverage: a summary can be perfectly balanced while covering very little of either side, or highly one-sided while covering a great deal of one side and none of the other, and the two metrics are reported side by side rather than combined into a single number precisely so that this distinction remains visible.

5.3.4 BERTScore F1 and the Independent Reference Summary

The three metrics above all score the summary against its own retrieved arguments. BERTScore F1 [zhang2020bertscore] instead scores it against an independently generated reference summary, giving UC2 one metric that is not itself computed from the same source material the summary was asked to be faithful to. BERTScore matches each candidate-summary token to its most similar reference-summary token by contextual embedding cosine similarity, and symmetrically each reference token to its most similar candidate token, and reports the F1 of the two resulting matching scores. Because the matching is over embeddings rather than surface tokens, a candidate and reference that paraphrase the same content in entirely different words still score highly, which is the property that makes BERTScore suitable here: two summaries of the same debate, written by different models, are unlikely to share much vocabulary even when they agree on substance.

The reference summary itself has to be constructed carefully to avoid a specific circularity. If the reference were generated by the same model used as the candidate summarizer, Qwen3:8b, BERTScore would partly reward stylistic self-similarity, a summary would score well for sounding like something the same model tends to produce, rather than for capturing the same content a genuinely different model would independently converge on. This thesis therefore generates the reference summary with a model held distinct from both the candidate summarizer and the LLM used for Faithfulness judging in Section ??, following the same disjoint-model reasoning already used for the dual-annotator ground truth (Section 4.7.1) and motivated by the same documented self-preference risk in LLM-based evaluation [panickssery2024selfpref]. A further, less obvious design choice is that the reference summary is regenerated separately for each pool condition and each system’s own retrieved argument set, rather than produced once as a single global gold reference reused everywhere. Given a universal reference, BERTScore would conflate

two different questions, whether a system’s retrieval enabled a summary close to one fixed target, and whether the candidate summarizer faithfully rendered whatever it was actually given, since a system that retrieves a different set of arguments than the reference was built from is penalized for that difference regardless of how well it summarized its own retrieval. Generating the reference from the same retrieved arguments the candidate summary was built from isolates the second question instead: BERTScore then measures whether the candidate summarizer’s rendering of a given input diverges from a stronger model’s rendering of the identical input, a generation-consistency check rather than a retrieval-quality-via-reference check, and the two are not the same claim.

5.3.5 Why UC2 Omits Holistic Rubric Judging

UC1’s rubric judge (Section ??) is retained as a useful supplementary diagnostic because it shows meaningful spread across systems and axes, evidence that it is discriminating between retrieved sets of genuinely different quality rather than defaulting to a uniform response. UC2’s version of the same judge, an independent 1–5 score across coverage, balance, insight, and specificity, scored by Llama 3.3 70B at temperature 0 on the same direct-scoring design that replaced UC1’s abandoned pairwise comparison (Section ??), was computed during evaluation-protocol development and did not show the same spread. Every system’s mean score on every axis clustered inside a narrow band near the top of the 1–5 scale, typically between 4.0 and 5.0, regardless of which retrieval system had produced the summary being judged. A metric that assigns nearly every summary a score within one point of the ceiling carries almost no information for ranking systems against each other: whatever genuine quality differences exist between summaries produced from different retrievals, the rubric judge is not resolving them.

This thesis therefore follows the underlying conference paper’s decision and reports UC2’s rubric judge as a supplementary, non-primary diagnostic, in favor of the four automatic metrics of Sections ??–??. Each of which has a narrower, more verifiable failure mode than a single holistic scalar and, unlike the rubric judge, demonstrably separates systems in practice. This is a deliberate asymmetry with UC1 rather than an inconsistency: the same judging methodology behaves differently on the two use cases because UC1’s retrieved sets are heterogeneous enough for a holistic judge to distinguish, while UC2’s five-field structured summaries, produced from a fixed prompt template regardless of which system fed them, apparently converge in surface quality closely enough that a coarse 1–5 scale cannot tell them apart even where the automatic metrics still can.

5.4 Reporting Conventions and Metric Reliability

Every metric defined in this chapter is computed at the level of a single (policy, stance) pair for UC1 or a single policy for UC2, then aggregated in two stages before it reaches

a comparison table in Chapter ?? . For UC1’s stance-aware metrics, a PRO score and a CON score are first computed independently for each policy and averaged into one per-policy figure, so that a policy with an unusually strong PRO side and a weak CON side, or vice versa, does not silently favor whichever stance a system happens to handle better. Per-policy figures are then averaged, together with their standard deviation, across the fixed 30-policy stratified sample of Section 4.5. The per-policy figures themselves are retained in the full evaluation report rather than discarded once the aggregate is computed, both because Chapter ?? cross-references them against policy-level covariates such as argument-count density and EQUIVALENT-community modularity, and because retaining a per-unit breakdown rather than only a pooled mean is standard practice in comparing IR systems across a fixed set of topics, where paired, per-topic comparison is generally more informative than comparing aggregate averages alone [smucker2007significance, demsar2006statistical].

Two further reporting conventions follow directly from decisions already fixed in Chapter 4. Every metric is reported separately under the native pool and the hardened pool, since Section ?? and Section ?? have already shown that a metric’s behavior, and in EILD’s case its very validity as a diversity measure, can differ qualitatively between the two conditions. And every UC1 metric is reported at the fixed $k = 5$ operating point as the primary comparison, with the pool-size and k sweep of Chapter ?? reported separately as a generalization check rather than folded into the same table, since a conclusion that only holds at one specific operating point is a materially weaker claim than one that holds across a swept range.

A metric’s reported value is only as trustworthy as its stability across independent evaluation runs, a question that matters most for the Random baseline, since a random retriever’s score on any given evaluation run is, by construction, one draw from a distribution rather than a deterministic property of the system. As a validation step taken during evaluation-protocol development, Random’s retrieval was re-run over 20 independent random seeds at the native-pool, $k = 5$ operating point, and nDCG@5 was recomputed on each. The resulting distribution has a mean of 0.388 and a 95% bootstrap confidence interval [efron1993bootstrap] of [0.377, 0.399], a width of roughly 0.022 around a value close to the single-seed figure reported as Random’s headline score in Chapter ?? . An interval this narrow, and this far from zero, indicates that Random’s elevated apparent competitiveness under the native pool (Section 4.7.4) is a stable, structural property of the native pool’s high relevance density, not an artifact of one unusually favorable random draw. This distinction matters directly for how Chapter ?? interprets any comparison involving Random: a tie between Random and a designed retrieval system under the native pool, given this check, is evidence that the metric’s headroom is compressed on that pool, an interpretation this thesis returns to when discussing why the pool was hardened in the first place, rather than evidence that either system’s evaluation run happened to be lucky. This kind of seed-level robustness check is one instance of a broader concern in constructing retrieval benchmarks that are difficult enough, and stable enough, to actually discriminate between systems rather than being dominated by

evaluation noise [**thakur2025freshstack**].

Appendix A

Appendix A

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Appendix B

Appendix B

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Bibliography

- [1] Xabier E. Barandiaran, Antonio Calleja-López, Arnau Monterde, and Carol Romero. *Decidim, a Technopolitical Network for Participatory Democracy*. Open access. Springer, 2024 (cit. on p. 1).
- [2] Bokyoung Shin, Jacqueline Floch, Mikko Rask, Peter Bæck, Christopher Edgar, Aleksandra Berditchevskaia, Pierre Mesure, and Matthieu Branlat. “A systematic analysis of digital tools for citizen participation”. In: *Government Information Quarterly* 41 (Sept. 2024), p. 101954. DOI: 10.1016/j.giq.2024.101954 (cit. on p. 1).
- [3] Miguel Arana-Catania, Felix-Anselm Van Lier, Rob Procter, Nataliya Tkachenko, Yulan He, Arkaitz Zubiaga, and Maria Liakata. “Citizen Participation and Machine Learning for a Better Democracy”. In: *Digital Government: Research and Practice* 2.3 (2021), pp. 1–22. ISSN: 2639-0175. DOI: 10.1145/3452118. URL: <http://dx.doi.org/10.1145/3452118> (cit. on p. 1).
- [4] Marco Lippi and Paolo Torroni. “Argumentation Mining: State of the Art and Emerging Trends”. In: *ACM Transactions on Internet Technology* 16.2 (2016), 10:1–10:25. DOI: 10.1145/2850417. URL: <https://doi.org/10.1145/2850417> (cit. on p. 3).
- [5] Aidan Hogan et al. “Knowledge Graphs”. In: *ACM Computing Surveys* 54.4 (2021), pp. 1–37. URL: <http://dx.doi.org/10.1145/3447772> (cit. on p. 3).
- [6] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. 2020, pp. 9459–9474 (cit. on pp. 4, 5).
- [7] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. “From Local to Global: A Graph RAG Approach to Query-Focused Summarization”. In: *arXiv preprint arXiv:2404.16130* (2024) (cit. on pp. 4–6).
- [8] Yunfan Gao et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2024. arXiv: 2312.10997 [cs.CL]. URL: <https://arxiv.org/abs/2312.10997> (cit. on p. 5).

- [9] Jaime Carbonell and Jade Goldstein. “The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries”. In: *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '98)*. ACM, 1998, pp. 335–336 (cit. on pp. 6, 20).
- [10] Sofiane Elguendouze, Lucas Anastasiou, Erwan Hain, Elena Cabrio, Anna De Liddo, and Serena Villata. “AM4DSP: Argumentation Mining in Structured Decentralized Discussion Platforms for Deliberative Democracy”. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 2025, pp. 796–805 (cit. on p. 8).
- [11] Lucas Anastasiou and Anna De Liddo. “BCause: Human-AI Collaboration to Improve Hybrid Mapping and Ideation in Argumentation-Grounded Deliberation”. In: arXiv preprint arXiv:2505.03584. 2025 (cit. on p. 8).
- [12] Maike Behrendt, Stefan Sylvius Wagner, Carina Weinmann, Marike Bormann, Mira Warne, and Stefan Harmeling. “Machine Learning for Enhancing Deliberation in Online Political Discussions and Participatory Processes: A Survey”. In: *arXiv preprint arXiv:2506.02533* (2025) (cit. on p. 8).
- [13] Moritz Plenz, Philipp Heinisch, Anette Frank, and Philipp Cimiano. “PAKT: Perspectivized Argumentation Knowledge Graph and Tool for Deliberation Analysis”. In: *arXiv preprint arXiv:2404.10570* (2024) (cit. on p. 9).
- [14] Simone Vagnoni and Víctor Rodríguez-Doncel. “A Deliberation Knowledge Graph: Bridging Institutional and Civic Democratic Discourse”. In: *Electronic Government and the Information Systems Perspective (EGOVIS 2025)*. Vol. 16049. Lecture Notes in Computer Science. Springer, 2025, pp. 122–136 (cit. on p. 9).
- [15] “DebateKG: Automatic Policy Debate Case Creation with Semantic Knowledge Graphs”. In: *Proceedings of the 4th New Frontiers in Summarization Workshop (NewSumm 2023)*. 2023 (cit. on p. 9).
- [16] George Papageorgiou, Vangelis Sarlis, Manolis Maragoudakis, and Christos Tjortjis. “Hybrid Multi-Agent GraphRAG for E-Government: Towards a Trustworthy AI Assistant”. In: *Applied Sciences* 15.11 (2025), p. 6315 (cit. on p. 10).
- [17] Michael Henry Tessler et al. “AI Can Help Humans Find Common Ground in Democratic Deliberation”. In: *Science* 386.6719 (2024), eadq2852 (cit. on p. 10).

Dedications

Dedications for everyone